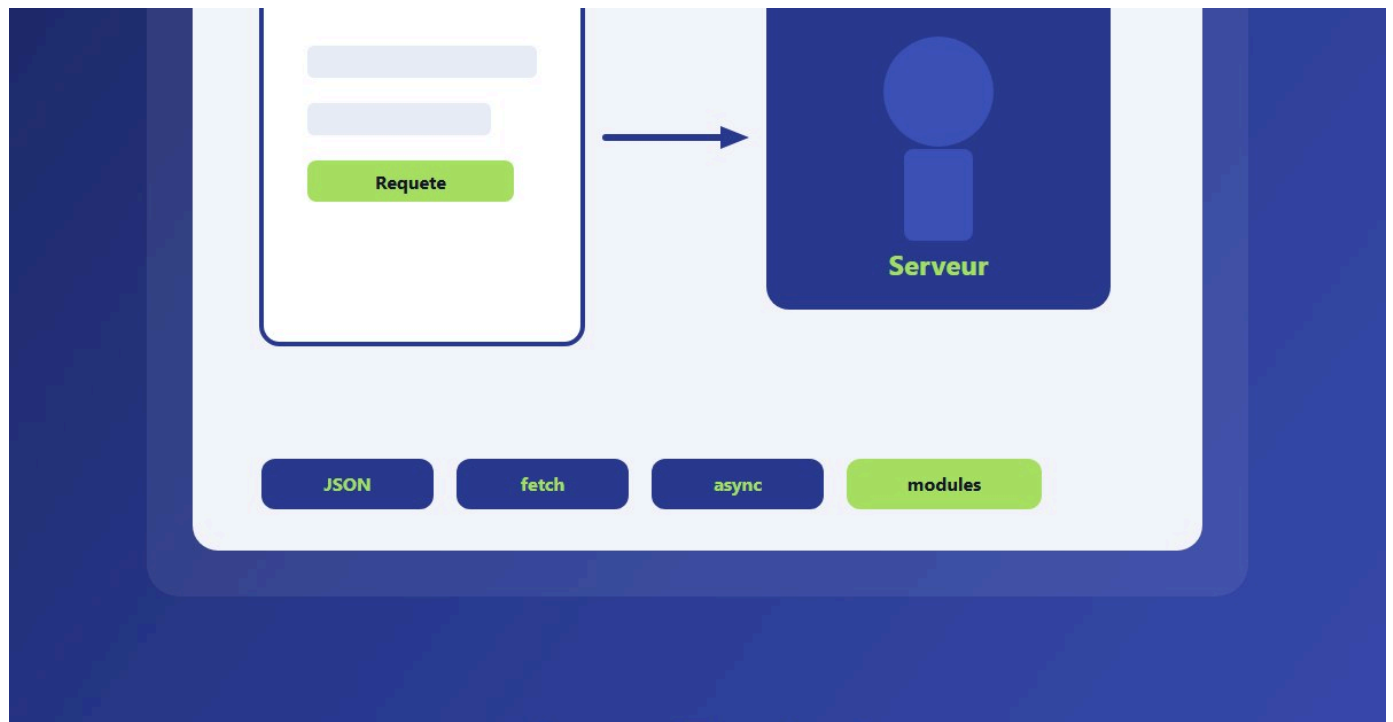




FORMATION INFORMATIQUE - SUITE

JavaScript - La suite

Formulaires, JSON, fetch, async/await et modules : la suite solide apres les bases.

DanielCraft - Livre debutant

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. [Chapitre 1 - Rappel rapide et carte du livre](#)

- [Ce que tu gardes en tete](#)
- [Ce que tu vas savoir faire](#)
- [Comment lire ce livre](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom DanielCraft : le fil rouge technique](#)
- [Ce que ce livre n'est pas](#)
- [Tableau rapide des chapitres](#)
- [Petite scene DanielCraft](#)

2. [Chapitre 2 - Formulaires qui tiennent la route](#)

- [Lire les champs](#)
- [Valider sans etre mechant](#)
- [HTML5 aide, JS decide](#)
- [Petite histoire](#)
- [Cas concret : todo avec garde-fou](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

3. [Chapitre 3 - JSON, le format des donnees voyageuses](#)

- [A quoi ca ressemble](#)
- [De texte vers objet : parse](#)
- [D'objet vers texte : stringify](#)
- [Pieges courants](#)
- [Exemple concret : todo en texte](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

4. [Chapitre 4 - fetch GET : demander des infos](#)

- [Premier appel](#)
- [Afficher dans la page](#)
- [Meteo : autre exemple mental](#)
- [Ce que fetch renvoie vraiment](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

5. [Chapitre 5 - Les promesses, sans jargon opaque](#)

- [then : quand ca marche](#)
- [catch : quand ca casse](#)
- [Pourquoi pas juste "attendre" ?](#)
- [Chainer sans se noyer](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

6. [Chapitre 6 - async/await : du fetch lisible](#)

- [Le geste de base](#)
- [Reecrire l'exemple meteo](#)
- [Attention : await seulement dans async](#)
- [try/catch avec await](#)
- [Quand garder then ?](#)
- [Erreur classique](#)
- [Petite histoire](#)
- [En vrai](#)
- [A toi](#)

7. [Chapitre 7 - Erreurs reseau et reponses foireuses](#)

- [response.ok](#)
- [try/catch async, version complete](#)
- [Différents types d'echecs](#)
- [Ne mens pas a l'utilisateur](#)
- [Cas meteo](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

8. [Chapitre 8 - fetch POST : envoyer un peu de JSON](#)

- [Anatomie d'un POST](#)
- [Brancher sur un formulaire](#)
- [GET vs POST en une phrase](#)
- [Ce que le serveur renvoie](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [Zoom : ce que tu envoies vraiment](#)
- [A toi](#)

9. [Chapitre 9 - Modules : decouper le code](#)

- [Un exemple simple](#)
- [Pieges a connaitre](#)
- [Petite histoire](#)
- [Erreur classique](#)

- [En vrai](#)
- [A toi](#)
- [import vs export : le vocabulaire](#)
- [Cycle de vie module](#)
- [Zoom DanielCraft](#)

10. [Chapitre 10 - Organiser un petit projet](#)

- [La structure de base](#)
- [Responsabilites](#)
- [Noms clairs](#)
- [Une seule source de verite](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Versioning et sauvegarde](#)
- [Evolutive sans sur-ingenerier](#)
- [Checklist avant de dire "c'est fini"](#)

11. [Chapitre 11 - Deboguer mieux](#)

- [La console, ton amie](#)
- [Lire une stack trace](#)
- [Points d'arret \(breakpoints\)](#)
- [Hypotheses courtes](#)
- [Reseau et DOM](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Routine de debug en cinq minutes](#)
- [Erreurs fetch frequentes a reconnaitre](#)
- [Outils complementaires](#)

12. [Chapitre 12 - Mini-projet : liste depuis une API](#)

- [Ce que ce n'est pas](#)
- [Parcours en etapes](#)
- [Squelette de depart](#)
- [Variante meteo \(idee\)](#)
- [Qualite minimale attendue](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

13. [Chapitre 13 - A retenir \(carte mentale\)](#)

- [Les idees solides](#)
- [La boucle DanielCraft](#)

- [Ce que tu peux oublier](#)
- [Petite checklist de poche](#)
- [Personnages, meme logique](#)
- [Phrase talisman](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Note de rythme](#)
- [Pour aller plus loin sans te perdre](#)

14. [Chapitre 14 - Atelier : formulaire robuste](#)

- [Exercice 1 - Structure HTML \(10 min\)](#)
- [Exercice 2 - Ecouter submit \(10 min\)](#)
- [Exercice 3 - Validation par liste d'erreurs \(15 min\)](#)
- [Exercice 4 - Focus et reset \(10 min\)](#)
- [Livrable](#)
- [Criteres de reussite](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [Variante avancee](#)
- [En vrai](#)
- [Note de rythme](#)
- [A toi](#)

15. [Chapitre 15 - Atelier : fetch et afficher](#)

- [Preparation](#)
- [Exercice 1 - HTML et bouton \(5 min\)](#)
- [Exercice 2 - fetch async \(15 min\)](#)
- [Exercice 3 - Erreurs \(10 min\)](#)
- [Exercice 4 - Bonus recherche locale \(15 min optionnel\)](#)
- [Livrable](#)
- [Criteres de reussite](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [Note de rythme](#)
- [A toi](#)

16. [Chapitre 16 - Atelier : decouper en modules](#)

- [Les fichiers cibles](#)
- [Exercice 1 - Extraire api.js \(10 min\)](#)
- [Exercice 2 - Extraire afficher.js \(10 min\)](#)
- [Exercice 3 - main.js orchestrateur \(15 min\)](#)
- [Exercice 4 - Verifier les tailles \(5 min\)](#)
- [Livrable](#)
- [Criteres de reussite](#)

- [Bonus](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [Note de rythme](#)
- [A toi](#)

17. [Chapitre 17 - CORS, en une page claire](#)

- [Ce que tu peux faire en apprenant](#)
- [Ce qu'il ne faut pas croire](#)
- [Petite histoire](#)
- [Lire le message d'erreur](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Preflight : une precision utile](#)
- [Same-origin en pratique](#)
- [En resume DanielCraft](#)

18. [Chapitre 18 - Debounce : ne pas frapper trop vite](#)

- [Idee en code](#)
- [Filtre local vs fetch](#)
- [Quand l'utiliser](#)
- [Quand ne pas l'utiliser](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Debounce vs throttle \(aperçu\)](#)
- [Variante avec fonction nommée](#)
- [En resume](#)

19. [Chapitre 19 - Petites habitudes de performance](#)

- [Moins de travail inutile](#)
- [DOM : batch mental](#)
- [Reseau](#)
- [Code lisible d'abord](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Mesurer sans obsession](#)
- [Accessibilite et perf](#)
- [Priorites pour ce niveau](#)

20. [Quiz final](#)

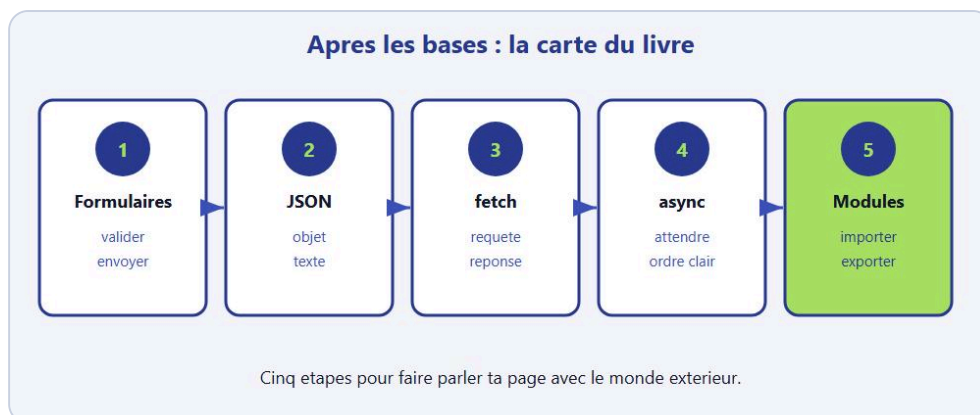
- [Questions](#)

- [Corriges](#)
- [Petite histoire](#)
- [Questions bonus \(auto-eval\)](#)
- [Note de rythme](#)
- [En vrai](#)
- [A toi](#)
- [Pour aller plus loin sans te perdre](#)

21. [Chapitre 21 - Bravo](#)

- [Ce que tu sais faire maintenant](#)
- [La suite possible](#)
- [Un dernier rappel](#)
- [Engagement 30 jours](#)
- [Petite scene DanielCraft](#)
- [En vrai](#)
- [A toi](#)
- [Note de rythme](#)
- [Zoom : le schema mental](#)

Chapitre 1 - Rappel rapide et carte du livre



Après les bases : formulaires, JSON, fetch, async, modules.

Tu as déjà parcouru les bases JavaScript. Variables, conditions, boucles, fonctions, DOM, événements : tout ça, tu l'as déjà touché. Ce livre ne repart pas de zéro. Il part du moment où ta page cesse d'être seule dans son coin et commence à parler avec le monde extérieur. Formulaires qui valident avant d'envoyer, données qui voyagent en **JSON**, requêtes réseau avec **fetch**, attente propre avec **promesses** et **async/await**, code découpé en **modules** : voilà le terrain de la suite. Tu vas sentir le passage entre "je fais bouger un bouton" et "je livre une petite feature qui charge et envoie pour de vrai".

Chez DanielCraft, on aime une image simple pour situer le niveau. Le premier livre JavaScript, c'était apprendre à faire bouger une pièce sur l'échiquier. Ici, c'est ouvrir la fenêtre, envoyer un message, attendre une réponse, ranger le code pour qu'il reste lisible dans six mois, et savoir expliquer à l'utilisateur quand ça charge ou quand ça casse. Tu n'as pas besoin d'un framework pour ça. React, Vue et les autres viendront plus tard si tu veux. Ici, on reste en JavaScript "nu", clair, dans le navigateur. Ce choix est volontaire : quand tu comprendras le flux à nu, un framework ne sera plus de la magie opaque.

Ce que tu gardes en tête

Tu sais sélectionner un élément avec `querySelector`. Tu sais écouter un clic ou un submit. Tu sais changer un texte avec `textContent`. Tu sais écrire une fonction. C'est déjà beaucoup. On ne revient pas sur `let`, `if`, `for`, ni sur "c'est quoi le DOM". Si ces mots te bloquent encore, relis le premier livre du parcours. Ici, on monte d'un cran sans te laisser en rade. Tu peux relire un chapitre du tome 1 en vingt minutes si un souvenir flanche : mieux vaut ça que de forcer en restant perdu.

Lea, freelance web, en est là : elle sait animer un menu et compter des clics. Max, artisan plombier qui code un peu son site, sait afficher ses tarifs. Sam, enseignant, fait des petits exercices interactifs pour ses élèves. Tous trois ont le même besoin maintenant : faire parler leur page avec des données réelles. Trois métiers, un même saut technique. Tu vas les retrouver tout au long du livre comme des compagnons de route, pas comme des mascottes inutiles.

Imagine une petite appli météo, un formulaire de contact, ou une liste de produits. Pour que ça marche vraiment, tu as besoin de plusieurs briques qui s'enchaînent. D'abord les formulaires : récupérer ce que la personne a tapé, vérifier, afficher une erreur claire si besoin. Ensuite JSON, le format texte standard pour transporter des données (nom, prix, température). Puis `fetch` : ta page demande des infos à un serveur.

Les promesses et `async/await` t'aident à attendre la réponse sans te perdre dans des pyramides de `then`. Les modules te permettent de découper le code en fichiers. Et à la fin, tu débogues, tu organises, tu fais des ateliers concrets.

On va souvent parler de trois exemples fil rouge. Une `todo` un peu plus sérieuse. Une météo qui charge des données. Un formulaire de contact qui valide avant d'envoyer. Ces exemples reviennent pour que tu sentes le progrès chapitre après chapitre, pas seulement des morceaux isolés. Quand une idée réapparaît sous un autre angle, ce n'est pas un oubli : c'est de l'ancrage.

♦ ASTUCE

Garde une page perso ouverte pendant toute la lecture (compteur, `todo`, galerie). Chaque chapitre te dira ce qu'il lui manque encore. Tu transformeras la lecture en checklist vivante.

Ce que tu vas savoir faire

Dans ce livre, tu vas valider des formulaires proprement, manipuler JSON avec `parse` et `stringify`, faire des requêtes GET et POST avec `fetch`, gérer les erreurs réseau et les codes HTTP, écrire du code `async` lisible, découper un projet en modules, déboguer avec méthode, comprendre **CORS** sans panique, ralentir une recherche avec `debounce`, et appliquer quelques réflexes de performance. Puis un mini-projet, un recap, trois ateliers, un quiz, et un bravo.

Niveau débutant solide qui a déjà codé un peu. Pas besoin d'être "tech avance". Besoin de curiosité et de patience : le réseau ne répond pas toujours, et c'est normal. Une appli qui explique l'échec vaut mieux qu'une appli qui marche seulement le jour du wifi parfait.

Comment lire ce livre

Lis dans l'ordre au début. Formulaires, JSON, `fetch`, promesses, `async` : ça s'enchaîne logiquement. Ensuite modules et organisation. Les ateliers sont là pour faire, pas seulement lire. Le quiz vérifie. Le dernier chapitre, c'est pour souffler et voir la suite. Tu peux revenir ensuite à un chapitre précis (erreurs réseau, CORS, `debounce`) comme à une fiche. À chaque fin de chapitre, il y a un "À toi". Fais-le. Cinq minutes valent mieux qu'une lecture passive.

Si un chapitre te semble déjà connu, lis-le quand même en diagonale : le ton et les pièges cités servent de filet de sécurité pour la suite. Les ateliers, eux, ne se "lisent" pas : ils se livrent.

Petite histoire

Lea avait un compteur de clics sur son portfolio. Ça marchait. Le client lui a demandé : "Et si on affichait mes produits depuis ma boutique en ligne ?" Lea a ouvert son fichier de 200 lignes, a cherché où mettre un `fetch`, et s'est sentie perdue. Ce livre répond exactement à ce moment : tu sais déjà bouger la pièce, maintenant tu apprends la route.

Max voulait un formulaire de devis sur son site. Sans validation, il recevait des mails vides. Sam voulait charger une liste de mots pour un quiz. Même schéma mental pour tous : lire, vérifier, envoyer ou afficher, gérer l'échec. Trois débuts différents, un même besoin de méthode.

Erreur classique

Croire que "je connais les bases" egal "je sais faire une vraie app". Les bases sont le moteur. La suite, c'est la route, le carburant, et la carte. Sans fetch et JSON, ta page reste seule dans sa bulle. Sans modules, ton fichier devient un spaghetti illisible. Sans validation de formulaire, tu envoies n'importe quoi au serveur ou tu frustres l'utilisateur avec des messages cryptiques.

Autre piege : vouloir tout faire d'un coup. Un formulaire qui POST, charge une liste, debounce une recherche et gere CORS le premier jour, c'est trop. Avance chapitre par chapitre. Chaque brique se pose sur la precedente.

⚠ ATTENTION

Ne saute pas les "A toi" pour "gagner du temps". Sans micro-action, le cerveau classe le chapitre comme "lu", pas comme "su". Cinq minutes actives battent quarante pages passives.

En vrai

Ouvre une page que tu as deja codee (compteur, todo simple, galerie basique). Note ce qui manque pour en faire quelque chose d'utile : charger une liste depuis ailleurs ? envoyer un message ? decouper le fichier en plusieurs morceaux ? Ce livre repond a ces questions une par une. Garde cette page ouverte en reference pendant la lecture. Si tu n'as aucune page sous la main, note trois idees de mini-apps et choisis-en une avant le chapitre 2.

A toi

Ecris en trois phrases ce que tu veux construire a la fin de ce livre. Pas un reseau social. Quelque chose de petit et concret : "afficher la meteo de ma ville", "valider un formulaire contact avant envoi", "charger une liste de produits depuis un fichier JSON". Garde ce but sur papier. On y reviendra au mini-projet et dans les ateliers.

Zoom DanielCraft : le fil rouge technique

Sur la duree du livre, tu vas assembler exactement ce type de flux : evenement utilisateur (clic, submit) -> validation locale -> fetch avec await -> verification response.ok -> JSON.parse implicite via .json() -> affichage DOM -> message d'erreur humain si echec. Ce schema revient partout. Apprends-le comme une recette de cuisine. Les ingredients changent (meteo, produits, citations), la recette reste. Quand tu seras bloque plus tard, reviens a cette recette avant de tout recrire.

Ce que ce livre n'est pas

Ce n'est pas un cours Node.js backend complet. Ce n'est pas un guide React. Ce n'est pas un manuel de securite avancee. Tu n'as pas besoin d'installer Webpack pour suivre. Tu as besoin d'un editeur, d'un navigateur, et d'un mini serveur local pour fetch et modules (Live Server ou python -m http.server suffisent). Si tu bloques la, relis le README du dossier livre ou demande de l'aide : ce setup revient dans tous les projets front modernes.

Tableau rapide des chapitres

Chapitres 2 a 8 : parler au reseau (formulaires, JSON, fetch, async, erreurs, POST). Chapitres 9 a 11 : ranger et deboguer. Chapitre 12 : mini-projet assemble. Chapitre 13 : recap. Chapitres 14 a 16 : ateliers mains dans le code. Chapitres 17 a 19 : CORS, debounce, perf legere. Chapitres 20 a 21 : quiz et bravo. Tu peux sauter temporairement 17-19 si tu es presse, mais reviens-y avant un vrai deploiement : CORS te rattrapera toujours.

Petite scene DanielCraft

Lea ouvre le chapitre 4 en sachant deja valider un formulaire. Max ouvre le chapitre 2 parce que son site envoie encore des champs vides. Sam fait le parcours complet avec ses eleves en douze seances. Trois rythmes, un meme livre. Choisis le tien, mais ne saute pas les "A toi" : c'est la que le livre devient competence.

★ A RETENIR

Bases = moteur. Suite = route + JSON + fetch + async + modules - avance brique par brique, livre les "A toi".

Chapitre 2 - Formulaires qui tiennent la route

Valider avant d'envoyer, expliquer l'erreur clairement.

Un formulaire, c'est une conversation entre ta page et la personne qui la visite. Elle écrit. Ta page écoute. Si quelque chose cloche, tu le dis clairement, sans agresser. Si tout va bien, tu continues vers l'envoi ou l'affichage d'un succès. Beaucoup de débutants font deux erreurs opposées : soit ils laissent le navigateur tout gérer sans rien expliquer, soit ils envoient le formulaire sans vérifier et la page se recharge en silence, effaçant tout ce que la personne avait tapé. Ici, on prend le contrôle avec JavaScript. Ce n'est pas "plus compliqué pour le plaisir" : c'est plus clair pour l'humain qui remplit.

Chez DanielCraft, on considère qu'un formulaire réussi, c'est un formulaire où l'utilisateur sait toujours où il en est. Champ manquant ? Message clair. Email bizarre ? Explication simple. Envoi en cours ? Indication visible. Succès ? Confirmation nette. Lea utilise ce réflexe sur chaque site client. Max l'applique à son formulaire de devis. Sam le montre à ses élèves comme base de toute appli web sérieuse. Tu vas retrouver ce même standard jusqu'au chapitre POST.

Imagine un guichet. La personne remplit une fiche. Avant de la glisser dans la boîte, un assistant vérifie : nom présent ? email avec un @ ? message assez long ? Si non, il renvoie la fiche avec un stylo sur la ligne à corriger. **preventDefault()**, c'est dire au navigateur : "Ne recharge pas la page, je gère la vérification moi-même." Sans ça, ta page clignote et le message disparaît. C'est le geste numéro un des formulaires en JS. Sans ce geste, le reste de ta logique n'a souvent même pas le temps de parler.

Lire les champs

Imagine un petit contact : nom, email, message. Rien d'exotique. Trois champs, une zone d'erreur, un bouton. C'est déjà assez pour apprendre le pattern qui servira partout.

```
<form id="contact">
  <label for="nom">Nom</label>
  <input id="nom" name="nom" type="text">

  <label for="email">Email</label>
  <input id="email" name="email" type="email">

  <label for="message">Message</label>
  <textarea id="message" name="message"></textarea>

  <p id="erreur" hidden></p>
  <button type="submit">Envoyer</button>
</form>
```

En JS, tu selectionnes le formulaire et tu ecoutes **submit**. Pas seulement le clic du bouton : Entree dans un champ compte aussi. C'est plus solide et plus accessible.

```
const form = document.querySelector("#contact");
const zoneErreur = document.querySelector("#erreur");

form.addEventListener("submit", function (event) {
  event.preventDefault();

  const nom = form.nom.value.trim();
  const email = form.email.value.trim();
  const message = form.message.value.trim();

  // on validera juste apres
});
```

La methode **trim()** enleve les espaces au debut et a la fin. Un champ rempli uniquement d'espaces, ce n'est pas un vrai contenu. Ce detail evite des bugs betes en production. Max a perdu des heures avec des "noms" faits d'espaces avant de comprendre ce reflexe.

✦ ASTUCE

Lis toujours les champs avec `.value.trim()` des le debut. Tu te proteges des faux remplissages et tu simplifies toutes les verifications suivantes.

Valider sans etre mechant

La **validation**, ce n'est pas punir. C'est aider. Un message utile dit quoi corriger et ou. Un message inutile dit juste "erreur" et laisse la personne deviner. Preferes une phrase humaine a un code mysterieux. Preferes pointer le champ concerne plutot que d'afficher un mur de texte en haut de page sans lien avec l'action.

```
function afficherErreur(texte) {
  zoneErreur.hidden = false;
  zoneErreur.textContent = texte;
}

function cacherErreur() {
  zoneErreur.hidden = true;
  zoneErreur.textContent = "";
}

form.addEventListener("submit", function (event) {
  event.preventDefault();
  cacherErreur();

  const nom = form.nom.value.trim();
  const email = form.email.value.trim();
  const message = form.message.value.trim();

  if (nom === "") {
    afficherErreur("Dis-nous ton nom, meme un prenom suffit.");
    form.nom.focus();
    return;
  }
});
```

```

if (email === "" || !email.includes("@")) {
  afficherErreur("L'email semble incomplet. Verifie le @.");
  form.email.focus();
  return;
}

if (message.length < 10) {
  afficherErreur("Ton message est un peu court. Ajoute quelques mots.");
  form.message.focus();
  return;
}

zoneErreur.hidden = false;
zoneErreur.textContent = "Merci " + nom + " ! Message pret a partir.";
});

```

Tu vois l'idée. On arrête dès la première erreur. On remet le **focus** sur le champ concerné. On explique en français simple. Chez DanielCraft, on préfère un message humain à un code d'erreur mystérieux copié depuis une doc API. Plus tard, tu pourras afficher plusieurs erreurs d'un coup (atelier 14) ; le principe reste le même : aider, pas intimider.

HTML5 aide, JS décide

Les attributs `required`, `type="email"`, `minlength` aident. Mais ne compte pas seulement dessus. Les navigateurs ne montrent pas tous les mêmes bulles d'erreur. Et parfois tu as des règles métier : le message doit mentionner un produit, la note doit être entre 1 et 5, le stock doit être un nombre positif. JS reste le chef d'orchestre. Tu peux combiner : HTML pour le minimum, JS pour affiner. Lea valide côté client pour le confort, et rappelle toujours qu'un vrai serveur devra revalider : le front aide, le back protège.

Exemple : Nom, Email, Message - valider avant Envoyer.

Petite histoire

Max avait mis un formulaire contact sur son site de plomberie. Les clients envoyaient des messages vides ou avec "@" oublié. Il recevait des mails inutilisables et perdait du temps à rappeler. En ajoutant `trim()`, `preventDefault()` et trois vérifications simples, il a divisé par deux les échanges inutiles. Lea, elle, valide côté client ET côté serveur : le front aide l'utilisateur, le back sécurise vraiment. Sam montre les deux niveaux à ses élèves pour qu'ils comprennent la différence. Trois façons de vivre le même problème, une même leçon : ne jamais faire confiance au "champ rempli" sans regarder.

Cas concret : todo avec garde-fou

Sur une todo, le piège classique c'est d'ajouter une tâche vide. Le pattern est exactement le même que pour le contact : lire, nettoyer, vérifier, agir.

```
const champ = document.querySelector("#tache");
const liste = document.querySelector("#liste");
const formTodo = document.querySelector("#form-todo");

formTodo.addEventListener("submit", function (event) {
  event.preventDefault();
  const texte = champ.value.trim();

  if (texte === "") {
    afficherErreur("Ecris une tâche avant d'ajouter.");
    return;
  }

  const li = document.createElement("li");
  li.textContent = texte;
  liste.appendChild(li);
  champ.value = "";
  champ.focus();
  cacherErreur();
});
```

Même logique partout : lire, nettoyer avec trim, vérifier, agir. Ce pattern reviendra au chapitre POST quand tu enverras le formulaire au serveur. Une fois que tu le sens dans les doigts, le reste du livre devient beaucoup plus fluide.

Erreur classique

Afficher l'erreur une seule fois, puis ne jamais la cacher. La personne corrige, renvoie, et l'ancien message reste affiché. Moche et confus. Efface l'erreur au début de chaque tentative, ou dès que la personne retape dans le champ concerné.

Autre piège : valider seulement au clic du bouton, jamais au submit. Si quelqu'un appuie sur Entrée dans un champ, ton code ne part pas. Écoute submit sur le formulaire entier. C'est plus solide et plus accessible.

En vrai

Ouvre un formulaire de site connu (inscription, contact, avis produit). Essaie d'envoyer vide. Regarde le message. Est-il clair ? Pointe-t-il le champ ? Note ce que tu aimes et ce qui t'agace. Tu peux voler les bonnes idées pour ton propre code. Compare aussi ce que fait le navigateur seul (required) et ce que fait le JS en plus. Tu verras vite pourquoi JS reste utile même avec HTML5.

A toi

Fais un formulaire "avis produit" : note de 1 à 5, commentaire. Refuse une note hors plage. Refuse un commentaire de moins de 5 caractères. Affiche un message de succès quand tout est bon. Pas besoin d'envoyer au serveur pour l'instant : concentre-toi sur la validation et les messages. Si ça marche, tu as la base solide pour le chapitre POST.

★ A RETENIR

Formulaire solide = preventDefault + trim + messages humains + focus sur le champ - ecoute submit, pas seulement le clic.

Chapitre 3 - JSON, le format des donnees voyageuses



JSON = donnees rangees en texte, lisibles par la machine.

Quand ta page parle a un serveur, elle n'envoie pas un objet JavaScript magique qui traverse l'internet tel quel. Elle envoie du texte. **JSON**, c'est la facon standard d'ecrire ce texte pour que tout le monde se comprenne : navigateur, serveur, API meteo, boutique en ligne, application mobile. JSON veut dire JavaScript Object Notation. Le nom fait peur. L'idée est simple : des donnees rangees comme des objets et des tableaux, mais ecrites en texte pur, lisible par une machine et par un humain. Une fois ce reflexe acquis, fetch devient beaucoup moins mysterieux.

Chez DanielCraft, on resume souvent ainsi : objet en memoire pour travailler, JSON en texte pour voyager. Lea manipule des fiches produit. Max envoie des demandes de devis. Sam prepare des listes de questions pour ses quiz. Dans tous les cas, le meme reflexe revient : **JSON.stringify** pour produire, **JSON.parse** pour consommer. Tu vas le refaire cent fois dans une vie de front : autant le rendre automatique des maintenant.

Imagine une valise etiquetee. Dedans, ce n'est pas l'objet lui-meme, c'est une fiche qui decrit l'objet : nom, prix, en stock oui ou non. Le transporteur (le reseau) ne transporte que la fiche. A l'arrivee, tu reconstruis l'objet en lisant la fiche. parse lit la fiche. stringify ecrit la fiche. Si la fiche est dechiree ou ecrite dans un autre langage, tu ne reconstruis rien de fiable. D'ou l'importance de verifier avant de parser.

A quoi ca ressemble

Voici une fiche produit en JSON. Tu peux la lire a voix haute : c'est presque du francais structure.

```
{
  "nom": "Casque audio",
  "prix": 59.9,
  "enStock": true,
  "tags": ["son", "promo"]
}
```

Et une liste :

```
[
  { "ville": "Paris", "temp": 18 },
  { "ville": "Lyon", "temp": 16 }
]
```

Tu reconnais les objets { }, les tableaux [], les chaines entre guillemets doubles, les nombres, true / false, et null. En JSON strict, les clefs sont toujours entre guillemets doubles. Pas de fonction. Pas de commentaire. Pas de virgule en trop apres le dernier element (certains parseurs tolerant, d'autres non). Ce formalisme n'est pas du snobisme : c'est ce qui permet a des outils differents de se comprendre sans negociation.

De texte vers objet : parse

Ta page recoit souvent une chaine de caracteres. Pour travailler avec, tu la transformes en vrai objet JS. Sans cette etape, tu ne peux pas faire data.nom ou data[0] de facon fiable.

```
const texte = '{"nom":"Casque audio","prix":59.9}';
const produit = JSON.parse(texte);

console.log(produit.nom); // Casque audio
console.log(produit.prix); // 59.9
```

JSON.parse lit le texte et construit l'objet. Si le texte est casse (virgule en trop, guillemets manquants, HTML d'erreur a la place du JSON), ca plante. On verra la gestion d'erreurs au chapitre reseau. Pour l'instant, retiens : parse sur du vrai JSON uniquement. Si tu as un doute, ouvre la reponse dans l'onglet Network et regarde si ca commence bien par { ou [.

D'objet vers texte : stringify

L'inverse : tu as un objet JS en memoire, tu veux l'envoyer ou le stocker. Le serveur ne "voit" pas ton objet vivant dans le navigateur. Il voit un corps de requete texte.

```
const contact = {
  nom: "Lea",
  email: "lea@exemple.fr",
  message: "Bonjour, je veux un devis."
};

const texte = JSON.stringify(contact);
console.log(texte);
// {"nom":"Lea","email":"lea@exemple.fr","message":"Bonjour, je veux un devis."}
```

Utile pour fetch en POST, pour localStorage, pour ecrire dans un fichier cote serveur. Quand Lea envoie un formulaire contact, stringify transforme l'objet { nom, email, message } en corps de requete que le serveur peut lire. Sans stringify, tu risques d'envoyer la fameuse chaine "[object Object]" - un classique douloureux.

⚠ ATTENTION

Ne confonds pas objet JS et JSON. En JS, les clefs peuvent parfois aller sans guillemets. En JSON strict, clefs et chaines sont en guillemets doubles. parse sur du HTML d'erreur = explosion garantie.

Pieges courants

Premier piege : confondre objet JS et JSON. En JS, tu peux ecrire { nom: "Lea" } avec des guillemets simples parfois, et parfois sans guillemets sur les clefs. En JSON vrai, les clefs et les chaines sont en guillemets doubles. Deuxieme piege : JSON.parse sur quelque chose qui n'est pas du JSON.

```
JSON.parse("salut"); // erreur
JSON.parse("{nom: Lea}"); // erreur (pas de guillemets sur les cles)
```

Troisième piège : croire que `stringify` garde les fonctions. Non. Les fonctions disparaissent. Les dates deviennent des chaînes ISO. Les `undefined` sautent souvent. JSON transporte des **données**, pas du comportement. Quatrième piège : parser deux fois ou stringifier deux fois.

```
const dejaTexte = '{"a":1}';
const encore = JSON.stringify(dejaTexte);
// resultat : une chaîne avec des échappements bizarres
```

Règle simple : parser quand tu reçois du texte JSON. Stringify quand tu veux produire du texte JSON. Une fois suffit. Si tu stringifies quelque chose qui est déjà une chaîne JSON, tu emballes du texte dans du texte : ça devient illisible pour le serveur.

Exemple concret : todo en texte

```
const taches = [
  { id: 1, titre: "Acheter du pain", faite: false },
  { id: 2, titre: "Appeler Lea", faite: true }
];

const pourStockage = JSON.stringify(taches);
// plus tard...
const relues = JSON.parse(pourStockage);
console.log(relues[0].titre); // Acheter du pain
```

Tu peux imaginer la même chose avec une réponse météo ou une liste de produits d'une API. Sam stocke ainsi ses listes de mots pour ses exercices. Max pourrait sauvegarder un brouillon de devis dans le navigateur. Le geste reste : objet → texte pour voyager ou stocker, texte → objet pour travailler.

Petite histoire

Lea a reçu une réponse API et a essayé `JSON.parse` dessus. Sauf que le serveur renvoyait une page HTML d'erreur 500, pas du JSON. Le parse a explosé. Elle a appris à vérifier `response.ok` avant de parser, et à regarder le `Content-Type` quand c'est disponible. Ce chapitre pose la base ; le chapitre erreurs réseau complète la protection. Sam montre souvent cette histoire à ses élèves pour qu'ils ne traitent jamais "toute réponse" comme du JSON sacré.

Erreur classique

Traiter toute réponse HTTP comme du JSON sans vérifier. Ou oublier `stringify` avant un POST et envoyer "[object Object]" au serveur. Ou copier-coller du JSON depuis un générateur en laissant une virgule finale : certains outils tolèrent, `JSON.parse` strict non. Ou stringifier deux fois "pour être sûr" et produire une chaîne échappée illisible.

En vrai

Ouvre la console du navigateur. Colle un petit `JSON.parse('{"ville":"Nantes","temp":14}')`. Affiche ville et temp. Puis `JSON.stringify` un objet de ton choix. Regarde le texte produit. C'est tout. Mais ce geste-la, tu vas le refaire cent fois dans ta vie de dev front. Fais-le maintenant pour que ce soit automatique. Si tu as un fichier `.json` local, ouvre-le aussi dans l'editeur et compare a ce que `parse` attend.

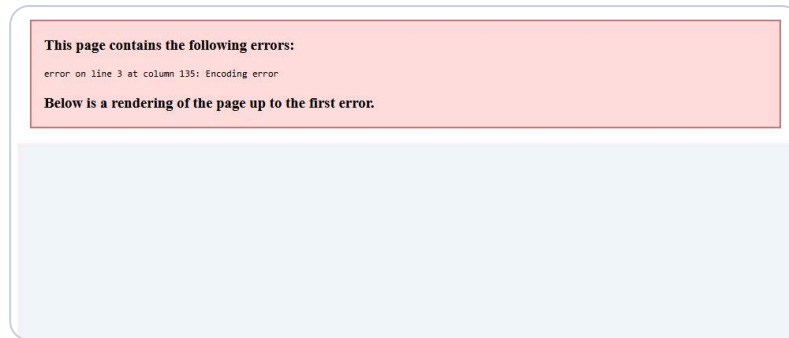
A toi

Cree un objet commande avec client (nom), produits (tableau de noms), et total (nombre). Transforme-le en texte JSON avec `stringify`. Reparse-le avec `parse`. Affiche une phrase : "Commande de X pour Y euros". Si ca marche sans erreur, JSON n'a plus de mystere pour toi. Passe ensuite au chapitre `fetch` pour voir ce texte voyager sur le reseau.

★ A RETENIR

JSON = texte de voyage. `parse` pour lire, `stringify` pour envoyer - une fois suffit, et seulement sur du vrai JSON.

Chapitre 4 - fetch GET : demander des infos



Ta page demande des infos a un serveur, puis affiche la reponse.

Jusqu'ici, tes donnees vivaient dans ton fichier ou dans le code que tu avais ecrit toi-meme. Maintenant, ta page peut demander des infos ailleurs : une API meteo, une liste de produits, un fichier JSON heberge sur un serveur, une base de donnees exposee via une URL. **fetch** est l'outil moderne du navigateur pour ca. En mode **GET**, le mode par defaut, tu dis simplement : "donne-moi cette ressource a cette adresse." C'est le geste qui transforme une page statique en page vivante.

Chez DanielCraft, on resume le flux en quatre mots : demander, attendre, lire, montrer. Lea charge les produits d'une boutique. Max affiche la meteo de sa ville sur son site. Sam recupere une liste de citations pour un exercice. Le geste est identique ; seules les URLs et les proprietes JSON changent. Une fois ce schema dans les doigts, tu pourras brancher n'importe quelle source compatible.

Ta page envoie une demande a une adresse (une **URL**). Le serveur repond. Tu lis la reponse. Souvent, la reponse est du JSON. Tu le parses, souvent avec **.json()**, puis tu affiches dans le DOM. C'est le coeur de beaucoup d'applications web simples : pages qui affichent des donnees vivantes sans rechargement complet. L'utilisateur clique, attend un instant, voit apparaitre une liste : la magie n'en est pas une, c'est ce flux-la.

Premier appel

Il existe des API publiques pour s'entrainer (JSONPlaceholder, Open-Meteo, etc.). L'exemple ci-dessous utilise une URL fictive pour rester clair. Le schema reste le meme partout. Ne te crispe pas sur l'adresse exacte : concentre-toi sur la chaine d'appels.

```
fetch("https://exemple.api/produits")
  .then(function (reponse) {
    return reponse.json();
  })
  .then(function (produits) {
    console.log(produits);
  });
```

Que se passe-t-il ? fetch part chercher l'URL. Quand la reponse HTTP arrive, le premier then recoit un objet **Response**. `reponse.json()` lit le corps et le transforme en objet ou tableau JS (comme un `JSON.parse` intelligent). Le second then recoit les donnees pretes a l'emploi. On detaillera les promesses au chapitre suivant. Pour l'instant, retiens la chaine : `fetch -> json -> utiliser`.

♦ ASTUCE

N'oublie jamais `return` devant `reponse.json()` dans un `then`. Sans `return`, le `then` suivant recoit `undefined` et tu cherches le bug pendant une heure.

Afficher dans la page

Imagine une liste de produits affichee a l'utilisateur. Pendant le chargement, tu montres un message. Ensuite tu vides la liste et tu ajoutes les elements un par un. Ce rythme "avant / pendant / apres" est deja une habitude pro.

```
<ul id="liste"></ul>
<p id="statut">Chargement...</p>
```

```
const liste = document.querySelector("#liste");
const statut = document.querySelector("#statut");

fetch("https://exemple.api/produits")
  .then(function (reponse) {
    return reponse.json();
  })
  .then(function (produits) {
    statut.textContent = produits.length + " produits trouves";
    liste.innerHTML = "";

    for (const p of produits) {
      const li = document.createElement("li");
      li.textContent = p.nom + " - " + p.prix + " EUR";
      liste.appendChild(li);
    }
  });
```

Tu vois le pattern. Pendant le chargement, tu montres un message. Ensuite tu vides la liste et tu ajoutes les elements un par un. Lea utilise exactement ce schema pour les catalogues clients. Simple, lisible, efficace. Plus tard, `async/await` rendra ce code encore plus lineaire, mais le flux mental reste le meme.

Meteo : autre exemple mental

Tu appelles une URL avec une ville en parametre. Tu recois `{ "ville": "Paris", "temp": 18, "ciel": "nuageux" }`. Tu ecris dans un paragraphe : "A Paris, il fait 18 degres, ciel nuageux." Memes etapes. Seules les proprietes et le texte affiche changent. Max a monte sa widget meteo locale avec ce principe en une soiree. Sam fait faire la meme chose a ses eleves avec des citations : meme ossature, autre contenu.

Ce que fetch renvoie vraiment

Attention : `fetch` reussit souvent meme si le serveur dit "404 introuvable" ou "500 erreur serveur". Techniquement, la requete reseau a abouti. Le code HTTP peut etre une erreur metier. On traitera `response.ok` au chapitre erreurs. Pour ce chapitre, concentre-toi sur le chemin heureux : URL correcte, JSON valide, affichage propre. Mais garde en tete que "pas d'exception" ne veut pas dire "tout va bien". C'est l'un des pieges les plus importants du livre.

⚠ ATTENTION

fetch qui "marche" (pas d'exception) ne veut pas dire que les données sont bonnes. Un 404 peut arriver sans planter. On corrigera ça avec `response.ok` au chapitre 7.



Exemple : Charger, attendre, puis afficher la liste.

Petite histoire

Sam voulait afficher des citations dans son exercice en ligne. Il a ouvert l'URL du fichier JSON dans le navigateur, a vu le texte brut, puis a écrit dix lignes de fetch. Les élèves ont vu les citations apparaître sans recharger la page. "C'est magique" ont-ils dit. Non : c'est fetch. La magie, c'est de comprendre le flux. Lea, elle, rappelle toujours d'ouvrir d'abord l'URL dans le navigateur : si le JSON n'est pas là, inutile de blâmer le JavaScript.

Erreur classique

Oublier `return reponse.json()` dans le `then`. Si tu écris seulement `reponse.json()` sans `return`, le `then` suivant reçoit `undefined`. Autre classique : traiter la réponse comme du JSON alors que c'est du HTML d'erreur. Ou appeler une mauvaise URL et croire que "fetch est cassé" alors que c'est l'adresse qui est fautive. Ou tester en ouvrant le fichier HTML en `file://` alors que fetch exige souvent un serveur local.

En vrai

Cherche une API publique simple ou prépare un fichier `produits.json` à côté de ta page. Ouvre l'URL dans le navigateur. Regarde le JSON brut. Puis tente un petit fetch dans la console ou dans un fichier servi par un mini serveur local (Live Server, `python -m http.server`). Le but : voir des vraies données arriver dans ta page. Si ça échoue en `file://`, ce n'est pas un échec personnel : c'est le navigateur qui protège.

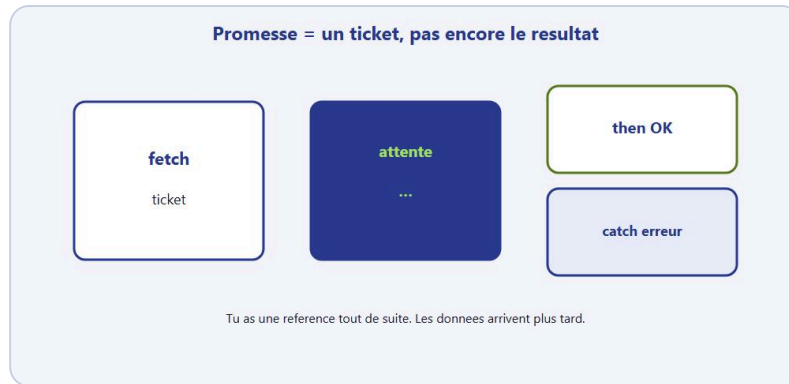
A toi

Écris une fonction `chargerProduits(url)` qui fait un GET, lit le JSON, et affiche chaque nom dans une liste `ul`. Même avec des données fictives en local (un fichier `produits.json` à côté), le geste compte. Si tu n'as pas encore de serveur local, écris le code quand même : tu le brancheras au chapitre `async/await` avec une syntaxe encore plus lisible.

★ **A RETENIR**

GET avec fetch = demander une URL, lire .json(), afficher - et se souvenir que "pas d'exception" n'est pas encore "succes".

Chapitre 5 - Les promesses, sans jargon opaque



Promesse = un ticket. then si OK, catch si erreur.

Quand tu appelles fetch, le resultat n'arrive pas tout de suite. Le reseau prend du temps. Le serveur peut etre loin, surcharge, ou simplement occupe a preparer la reponse. JavaScript dans le navigateur ne peut pas rester bloque a attendre comme un ascenseur en panne : la page doit rester utilisable. L'utilisateur doit pouvoir scroller, cliquer, taper pendant que ta requete voyage. C'est pour ca que le web moderne parle en "plus tard" plutot qu'en "attends ici jusqu'a la fin".

Une **promesse**, c'est un objet qui dit : "je te donnerai une valeur plus tard, ou une erreur." Tu branches ensuite ce qui se passe en cas de succes (**then**) ou d'echec (**catch**). C'est le langage du "plus tard" sur le web moderne. Tu n'as pas besoin de tout le jargon interne pour commencer. Tu as besoin de brancher correctement succes et echec.

Tu commandes une pizza. Le restaurant te donne un ticket. Ce ticket n'est pas la pizza. C'est la promesse de la pizza. Plus tard, soit tu recois la pizza (then), soit on t'annonce un probleme (catch). fetch te rend ce ticket tout de suite. Les donnees arrivent apres. Lea compare ca a un numero de suivi colis : tu as une reference immediate, le contenu arrive plus tard. Si tu ouvres le carton trop tot (code juste apres fetch sans then), tu trouves du vide.

Chez DanielCraft, on insiste : les promesses ne sont pas un exercice theorique de cours. Fetch, lecture de fichiers, timers avances, plein d'APIs modernes parlent en promesses. Une fois l'idee du ticket pizza digeree, async/await au chapitre suivant devient presque confortable. Tu ne changes pas de concept : tu changes de facon d'ecrire.

then : quand ca marche

```
fetch("https://exemple.api/meteo?ville=Paris")
  .then(function (reponse) {
    return reponse.json();
  })
  .then(function (data) {
    console.log("Temperature :", data.temp);
  });
```

Chaque then peut renvoyer une nouvelle valeur. Le then suivant la recoit. C'est pour ca qu'on ecrit return reponse.json() : **json()** renvoie aussi une promesse, parce que lire le corps de la reponse prend un peu de temps. Si tu oublies le return, la chaine se casse silencieusement. Ce "silence" est souvent plus dangereux qu'une erreur rouge : tu crois que tout va bien, et data est undefined.

catch : quand ca casse

```
fetch("https://exemple.api/meteo?ville=Paris")
  .then(function (reponse) {
    return reponse.json();
  })
  .then(function (data) {
    console.log(data.temp);
  })
  .catch(function (erreur) {
    console.log("Probleme :", erreur.message);
  });
```

Si le reseau tombe, ou si json() explose sur un texte invalide, tu tombes dans catch. Un seul catch a la fin peut couvrir toute la chaine. Pratique. Max a ajoute un catch sur sa meteo : quand le wifi de chantier est faible, la page affiche "Meteo indisponible" au lieu de rester vide. Sam insiste : un ecran qui explique vaut mieux qu'une console rouge invisible pour l'eleve.

♦ ASTUCE

Ajoute toujours un catch (ou un try/catch plus tard) des le premier fetch "serieux". Tester le chemin rate une fois te sauve des demos humiliantes.

Pourquoi pas juste "attendre" ?

Parce que JS dans le navigateur est **evenementiel**. Pendant que la requete voyage, l'utilisateur peut encore interagir. Les promesses permettent de dire "continue ta vie, et rappelle-moi quand c'est pret." Tu n'as pas besoin de connaitre tous les etats internes (pending, fulfilled, rejected) pour commencer. Retiens : une promesse aboutit ou echoue ; tu branches then et catch. Le reste du vocabulaire viendra si tu en as besoin.

Chainer sans se noyer

Evite les pyramides de then imbriquees. Preferes une ligne claire. Une fonction qui retourne la chaine reste lisible et reutilisable.

```
function afficherMeteo(ville) {
  return fetch("https://exemple.api/meteo?ville=" + encodeURIComponent(ville))
    .then(function (reponse) {
      return reponse.json();
    })
    .then(function (data) {
      document.querySelector("#meteo").textContent =
        "A " + data.ville + ", " + data.temp + " degres";
    })
    .catch(function () {
      document.querySelector("#meteo").textContent =
        "Impossible de charger la meteo.";
    });
}

afficherMeteo("Lyon");
```

encodeURIComponent protege les accents et espaces dans l'URL. Petit detail utile quand Lea passe "Saint-Etienne" ou "Aix-en-Provence" en parametre. Sans ca, certaines villes cassent silencieusement la requete.

⚠ ATTENTION

Le code juste apres fetch(...) s'execute tout de suite. Les donnees n'arrivent que dans then. Ecrire console.log juste apres fetch en croyant voir les donnees, c'est le piege numero un.

Petite histoire

Sam a ecrit un fetch sans catch. Un jour de cours, le fichier JSON etait mal nomme. La console est devenue rouge, les eleves ont panique, Sam aussi. En ajoutant un catch qui ecrit un message dans la page, le probleme est devenu visible et expliquable. "La liste est indisponible" vaut mieux qu'un ecran fige et une console effrayante. Lea raconte la meme chose aux juniors : un catch n'est pas du pessimisme, c'est du professionnalisme.

Erreur classique

Oublier le catch, puis ne rien voir quand ca rate. Ou mettre du code juste apres fetch(...) en croyant que les donnees sont deja la.

```
// FAUX reflexe
fetch(url);
console.log("deja arrive ?"); // non, trop tot
```

Le code apres fetch s'execute tout de suite. Les donnees arrivent dans then, plus tard. Autre piege : enchaîner trop de then sans return intermediaire. La chaine devient une cascade de undefined. Tu cherches alors un bug "dans les donnees" alors que le probleme est dans le branchement.

En vrai

Prends un fetch qui marche chez toi. Ajoute un catch qui ecrit un message dans la page. Coupe le wifi une seconde, ou mets une URL inventee, et regarde le catch se declencher. Tu dois sentir la difference entre succes et echec. C'est un reflexe pro. Si tu n'oses pas couper le wifi, une URL inventee suffit largement.

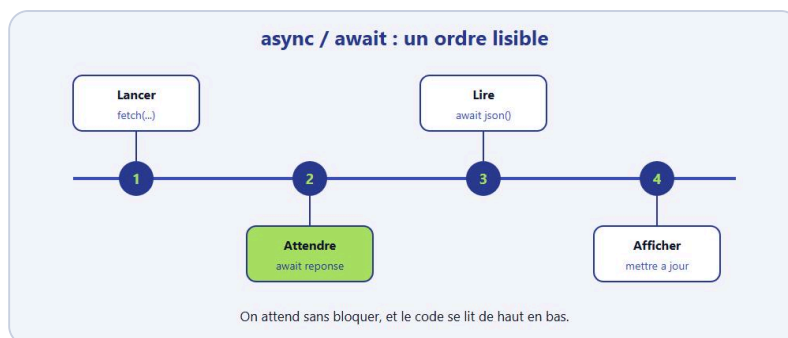
A toi

Ecris chargerTodos(url) avec then et catch. En succes, affiche le nombre de todos dans un paragraphe. En echec, affiche "Liste indisponible". Pas besoin que l'API soit parfaite : le branchement compte. Au chapitre suivant, tu reecriras la meme fonction en async/await pour comparer.

★ A RETENIR

Promesse = ticket "plus tard" : then pour le succes, catch pour l'echec, return dans la chaine - le code apres fetch n'a pas encore les donnees.

Chapitre 6 - async/await : du fetch lisible



async/await : attendre sans bloquer, dans un ordre lisible.

Les then marchent. Mais quand tu enchaines plusieurs etapes, ou quand tu lis ton propre code six mois plus tard, la chaine devient une chenille difficile a suivre. **async** et **await** permettent d'ecrire presque comme du code "normal", ligne apres ligne, tout en restant asynchrone. Tu gardes la page reactive, mais ton cerveau lit de haut en bas. Chez DanielCraft, on passe souvent en async/await des que le flux depasse deux etapes. Ce n'est pas un autre concept magique : c'est la meme histoire de promesse, mieux racontee.

Lea a reecrit ses chargeurs de produits en async/await et a divise par deux le temps de debug. Max, qui code un peu le soir, prefere cette syntaxe parce qu'elle ressemble a une recette de cuisine. Sam la montre apres les promesses pour que ses eleves voient que ce n'est pas un autre concept : c'est la meme histoire, mieux ecrite. Si tu as bien digere then/catch, ce chapitre devrait te faire sourire de soulagement.

Le geste de base

Tu marques une fonction avec async. Dedans, tu peux await une promesse. JavaScript attend le resultat de cette ligne, puis continue. La page, elle, n'est pas fige : d'autres evenements peuvent encore se produire.

```

async function chargerProduits() {
  const reponse = await fetch("https://exemple.api/produits");
  const produits = await reponse.json();
  console.log(produits);
}

chargerProduits();
  
```

Compare avec la version then du chapitre precedent. C'est la meme histoire, plus lisible. Demande. Lis JSON. Utilise. Le cerveau aime ca. Tu n'as plus a sauter mentalement d'un then a l'autre pour suivre le fil.

Reecrire l'exemple meteo

Version claire pour afficher la meteo d'une ville. Tu lis de haut en bas sans sauter entre then. Chaque await marque un point d'attente explicite.

```

async function afficherMeteo(ville) {
  const url = "https://exemple.api/meteo?ville=" + encodeURIComponent(ville);
  const reponse = await fetch(url);
  const data = await reponse.json();

  document.querySelector("#meteo").textContent =
    "A " + data.ville + ", il fait " + data.temp + " degres";
}

```

Quand Lea revoit ce code dans un an, elle comprend en dix secondes ce qu'il fait. C'est exactement le genre de lisibilité qu'on cherche dans un vrai projet, même petit.

Attention : await seulement dans async

Tu ne peux pas écrire `await fetch(...)` tout seul au milieu d'un script classique (sauf modules top-level dans certains contextes modernes, qu'on ne généralise pas ici). Encapsule dans `async function`. Aussi : une fonction `async` renvoie toujours une **promesse**. Si tu `return` produits, l'appelant reçoit une promesse de produits. Il devra `await` ou `.then` à son tour.

```

async function getProduits() {
  const reponse = await fetch("https://exemple.api/produits");
  return reponse.json();
}

async function demarrer() {
  const produits = await getProduits();
  console.log(produits.length);
}

demarrer();

```

Ce découpage est propre : `getProduits` parle au réseau, `demarrer` orchestre. Tu retrouveras ce pattern au chapitre modules. Séparer "qui charge" et "qui affiche" commence déjà ici, même dans un seul fichier.

⚠ ATTENTION

Oublier `await` est le piège classique : `const reponse = fetch(url)` te donne une `Promise`, pas une `Response`. Ensuite `reponse.json()` plante. Ou attendre `json()` sans `await` et te demander pourquoi `data` est encore une `Promise`.

try/catch avec await

Les erreurs réseau se détaillent au chapitre suivant. Mais le réflexe arrive déjà. **try/catch** autour d'`await`, c'est l'équivalent confortable de `.catch`. Même rôle, syntaxe plus familière pour qui vient de langages synchrones.

```

async function afficherMeteo(ville) {
  try {
    const reponse = await fetch(
      "https://exemple.api/meteo?ville=" + encodeURIComponent(ville)
    );
    const data = await reponse.json();
    document.querySelector("#meteo").textContent =
      data.ville + " : " + data.temp + " degres";
  } catch (erreur) {
    document.querySelector("#meteo").textContent =
      "Meteo indisponible pour le moment.";
  }
}

```

Tu gardes le meme confort de lecture que le chemin heureux, tout en protegeant le chemin rate. Au chapitre suivant, on ajoutera `response.ok` pour completer le fichier.

✦ ASTUCE

Quand tu encapsules plusieurs `await` dans un `try`, un seul `catch` couvre `fetch`, `json()` et souvent l'affichage. Commence large, puis affine si tu as besoin de messages differents.

Quand garder `then` ?

Parfois un tout petit enchainement. Parfois une lib qui renvoie des promesses et tu branches un seul `then`. Ce n'est pas interdit. Mais pour du code que tu lis et modifies regulierement, `async/await` gagne souvent. L'important : comprendre les deux, choisir selon la lisibilite. DanielCraft ne dogmatise pas : on choisit ce qui se relit le mieux dans six mois. `then`, c'est lire une recette en notes en marge ("quand la pizza arrive, fais ceci"). `async/await`, c'est lire la recette dans l'ordre avec des pauses marquees ("attends que le four chauffe, puis enfourne"). Le resultat final est le meme. L'experience de lecture change. Choisis l'outil qui te fait moins d'erreurs de lecture.

Erreur classique

Oublier `await` :

```

const reponse = fetch(url); // reponse = Promise, pas Response
const data = await reponse.json(); // plante : Promise n'a pas .json

```

Ou croire que `async` magique rend le code synchrone pour tout le monde. Non : ca rend ton ecriture lineaire. Le temps, lui, continue de passer. La page reste interactive. Ou mettre `await` dans une fonction non `async` : erreur de syntaxe immediate, heureusement. Ou oublier `await` sur `reponse.json()` et te demander pourquoi `data` est une Promise.

Petite histoire

Max a copie un tuto avec `await` sans `async` sur la fonction. Rien ne marchait. En ajoutant `async` devant fonction, tout s'est debloque. Lea, elle, a deja vu des juniors oublier `await` sur `reponse.json()` et se demander pourquoi `data` est une Promise. Ces deux oublis sont les plus frequents. Garde-les en tete. Sam les ecrit au tableau avant chaque atelier `fetch`.

En vrai

Reprends un fetch en then que tu as déjà écrit (chapitre 4 ou 5). Reecris-le en async/await. Compare les deux versions cote à cote. Laquelle te parle le plus ? Garde celle-la pour la suite du livre et pour tes projets perso. L'exercice n'est pas de "preferer la mode" : c'est de preferer ce que tu debogues plus vite.

A toi

Ecris async function chargerEtAfficher(url) qui charge une liste JSON et remplit un ul. Ajoute un message "Chargement..." avant le fetch, puis remplace-le par le resultat ou une erreur. Tu dois sentir le rythme : avant / pendant / apres. Ce rythme, c'est celui de toute appli web qui parle au reseau.

★ A RETENIR

async/await = meme promesse, lecture lineaire - await dans une fonction async, try/catch pour l'echec, ne jamais oublier await.

Chapitre 7 - Erreurs reseau et reponses foireuses



Message simple pour l'utilisateur, details pour toi.

Le reseau n'est pas gentil. Wifi coupe. Serveur en maintenance. URL tapee de travers. JSON invalide renvoie par erreur. Et le piege le plus vicieux pour les debutants : une reponse HTTP 404 ou 500 que fetch considere quand meme comme "arrivee". Techniquement, la requete a abouti. Metier, c'est un echec. Ce chapitre te donne un reflexe solide : verifier, attraper, expliquer a l'utilisateur. Chez DanielCraft, on considere qu'une appli sans gestion d'erreur n'est pas livrable, meme pour un exercice.

Lea a deja vu un client dire "ca marche chez moi" alors que la moitie des utilisateurs avait un ecran blanc en 4G faible. Max a appris a afficher "Envoi impossible" au lieu de faire semblant. Sam montre a ses eleves que gerer l'echec fait partie du metier, pas un bonus optionnel.

response.ok

Quand fetch se termine sans planter, tu as un objet Response. Regarde `reponse.ok` : c'est true pour les codes 200-299. Sinon, quelque chose cloche cote serveur ou URL.

```
async function chargerProduits(url) {
  const reponse = await fetch(url);

  if (!reponse.ok) {
    throw new Error("HTTP " + reponse.status);
  }

  return reponse.json();
}
```

Tu transformes un "mauvais statut" en vraie erreur. Ensuite try/catch peut la recuperer. C'est le pattern que DanielCraft recommande presque toujours avec fetch. Sans ce if, tu risques d'afficher "Succes" avec des donnees vides ou du HTML d'erreur parse en JSON.

try/catch async, version complete

```
const statut = document.querySelector("#statut");
const liste = document.querySelector("#liste");

async function afficherProduits() {
  statut.textContent = "Chargement...";
  liste.innerHTML = "";
```



```

try {
  const reponse = await fetch("https://exemple.api/produits");

  if (!reponse.ok) {
    throw new Error("Le serveur a repondu " + reponse.status);
  }

  const produits = await reponse.json();

  if (!Array.isArray(produits)) {
    throw new Error("Format inattendu");
  }

  statut.textContent = produits.length + " produits";
  for (const p of produits) {
    const li = document.createElement("li");
    li.textContent = p.nom;
    liste.appendChild(li);
  }
} catch (erreur) {
  statut.textContent = "Impossible de charger la liste. Reessaie plus tard.";
  console.error(erreur);
}

}

afficherProduits();

```

Tu vois les couches. Message pendant le chargement. Verification HTTP. Verification du format (tableau attendu). Affichage. Et si ca casse, un message humain a l'ecran plus le detail technique dans la console pour toi, le dev.

Differents types d'echecs

Coupure reseau : fetch rejette, tu tombes dans catch. 404 ou 500 : fetch resout, mais ok est false. D'ou le if (!reponse.ok). JSON casse : reponse.json() rejette. Encore le catch. Bug dans ton affichage : aussi le catch, si tu as enveloppe assez large. Parfois trop large : apprends a lire console.error pour voir la vraie cause sans deviner.

Ne mens pas a l'utilisateur

"Erreur 500 Internal Server Error" ne parle qu'aux devs. Preferes "Le service produits est indisponible. Reessaie dans cinq minutes." Tu peux garder le detail pour la console ou un mode debug. Pour un formulaire de contact, "Envoi impossible (reseau)" vaut mieux qu'un ecran blanc ou un spinner infini. Ton appli a deux publics : l'utilisateur (message simple) et toi (console avec details). L'utilisateur veut savoir quoi faire. Toi, tu veux savoir pourquoi. Ne confonds pas les deux. Max affiche "Service indisponible" au client et garde le stack trace pour lui.

Cas meteo

```

async function meteo(ville) {
  try {
    const reponse = await fetch(
      "https://exemple.api/meteo?ville=" + encodeURIComponent(ville)
    );
    if (!reponse.ok) {
      throw new Error("ville ou service introuvable");
    }
  }
}

```

```

    }
    const data = await reponse.json();
    return data;
  } catch (e) {
    return null;
  }
}

async function ui() {
  const data = await meteo("Toulouse");
  const zone = document.querySelector("#meteo");
  if (!data) {
    zone.textContent = "Pas de meteo pour cette ville.";
    return;
  }
  zone.textContent = data.temp + " degres - " + data.ciel;
}

```

Renvoyer null puis tester, c'est une autre façon de gérer. L'important : ne jamais faire comme si les données étaient là quand elles ne le sont pas.

Erreur classique

Afficher "Succes !" alors que ok est false, parce que tu n'as regardé que "pas d'exception". Ou avaler l'erreur avec un catch vide. Un catch vide, c'est cacher la poussière sous le tapis. Ou afficher le message technique brut à l'utilisateur final : effrayant et inutile.

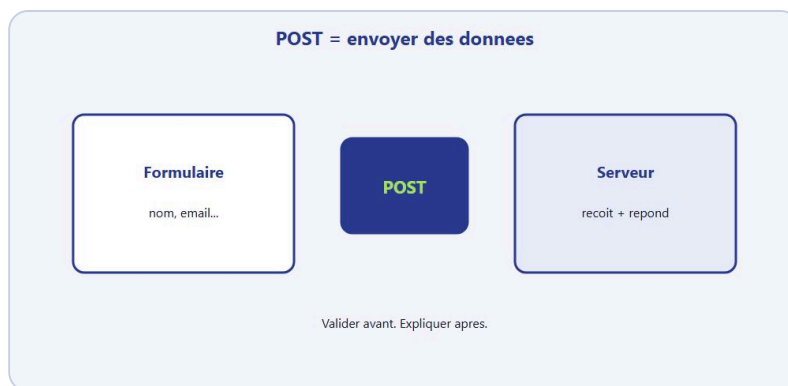
En vrai

Force une erreur volontairement. URL inventée. Puis une URL qui existe mais renvoie 404. Observe la différence entre rejet réseau et ok === false. Écris les deux cas dans un petit commentaire pour toi. Ce sera ta fiche perso.

A toi

Prends ton chargeur de liste (chapitre 4 ou 6). Ajoute response.ok, un vrai message utilisateur, et un console.error. Teste le chemin heureux et le chemin rate. Les deux doivent "marcher" : l'un affiche des données, l'autre explique le problème sans planter la page.

Chapitre 8 - fetch POST : envoyer un peu de JSON



POST : envoyer les donnees du formulaire au serveur.

GET, c'est "donne-moi". POST, c'est "voici quelque chose". Formulaire de contact, creation d'une todo, envoi d'un avis produit, enregistrement d'une commande : souvent, tu POSTES un corps JSON au serveur. On reste leger dans ce chapitre. Pas besoin d'authentification complexe ni de tokens OAuth. Juste : methode, headers, body. Le trio qui fait voyager tes donnees du navigateur vers le serveur.

Chez DanielCraft, le flux contact est un classique : valider cote client (chapitre 2), montrer "Envoi en cours", POST le JSON, gerer succes et echec. Lea l'implemente sur presque chaque site vitrine. Max l'utilise pour son formulaire de devis. Sam simule l'envoi en exercice avant de brancher un vrai backend.

Anatomie d'un POST

```
async function envoyerContact(contact) {
  const reponse = await fetch("https://exemple.api/contact", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(contact)
  });

  if (!reponse.ok) {
    throw new Error("Envoi refuse (" + reponse.status + ")");
  }

  return reponse.json();
}
```

Trois ingredients essentiels. method: "POST" pour dire que tu envoies des donnees. Content-Type: application/json pour prevenir que le corps est du JSON. body avec le texte produit par JSON.stringify. Sans le header, certains serveurs ne comprennent pas. Sans stringify, tu envoies "[object Object]" et ca finit tres mal.

Brancher sur un formulaire

```
const form = document.querySelector("#contact");
const statut = document.querySelector("#statut");

form.addEventListener("submit", async function (event) {
```

```

event.preventDefault();

const contact = {
  nom: form.nom.value.trim(),
  email: form.email.value.trim(),
  message: form.message.value.trim()
};

if (contact.nom === "" || contact.message.length < 10) {
  statut.textContent = "Complete le formulaire avant d'envoyer.";
  return;
}

statut.textContent = "Envoi en cours...";

try {
  const resultat = await envoyerContact(contact);
  statut.textContent = "Message envoye. Merci !";
  form.reset();
  console.log(resultat);
} catch (e) {
  statut.textContent = "Envoi impossible. Verifie ta connexion.";
  console.error(e);
}
});

```

Tu valides d'abord (chapitre 2). Tu montres un etat intermediaire. Tu postes. Tu geres succes et echec. C'est un flux realiste de page contact. Note le async sur le handler submit : indispensable pour utiliser await dedans.

GET vs POST en une phrase

GET pour lire sans modifier (liste, meteo, detail produit). POST pour creer ou envoyer (message, nouvelle tache, commande). Il existe aussi PUT, PATCH, DELETE pour mettre a jour ou supprimer. Plus tard. Pour ce livre, GET plus POST suffisent largement pour des mini-apps utiles.

Ce que le serveur renvoie

Parfois rien d'utile (code 204). Parfois un objet { "id": 42, "status": "recu" }. Parfois une erreur JSON { "erreur": "email invalide" }. Lis la doc de l'API quand tu en as une. En exercice, tu peux simuler avec un service de test ou un petit backend maison. L'important cote front : savoir envoyer proprement et lire la reponse.

Petite histoire

Lea a oublie JSON.stringify une fois. Le serveur recevait "[object Object]". Le client jurait que "le formulaire est casse". En regardant l'onglet Network du navigateur, elle a vu le corps de la requete. Deux minutes pour corriger, des heures de debug evitees ensuite. Verifie toujours ce qui part vraiment sur le fil.

Erreur classique

Ecrire body: contact au lieu de body: JSON.stringify(contact). Ou oublier async sur le handler submit et mettre await dedans. Ou poster sans preventDefault et perdre l'etat au rechargement. Ou croire que POST "marche" parce que le reseau n'a pas plante, sans lire reponse.ok. Meme regle qu'au chapitre 7.

En vrai

Si tu as un endpoint de test (style JSONPlaceholder posts), envoie un petit objet { titre, corps }. Affiche l'id renvoyé. Sinon, écris la fonction et un `console.log(JSON.stringify(contact))` pour vérifier le corps avant même d'appeler le réseau. Voir le JSON avant l'envoi, c'est une habitude pro.

Zoom : ce que tu envoies vraiment

Ouvre Network avant de cliquer. Regarde Headers (Content-Type) et Payload (le body). Si tu vois [object Object], `stringify` a raté. Si tu vois du JSON clair, tu es bon. Lea fait ce geste à chaque nouveau formulaire. Max aussi, depuis qu'il a envoyé un devis "casse" sans le savoir. Sam le montre en projecteur : silence, puis "ah". Chez DanielCraft, voir le fil avant de déboguer le serveur, c'est un réflexe.

A toi

Ajoute un bouton "Envoyer l'avis" avec note et commentaire. Valide les champs. POST en JSON vers une URL de test ou simule avec `console.log`. Affiche "Avis enregistré" ou un message d'erreur. Garde le code court. La clarté bat la longueur. Bonus : désactive le bouton pendant l'envoi pour éviter le double POST.

Chapitre 9 - Modules : decouper le code



Un fichier = une responsabilite. import / export.

Quand ton fichier app.js fait 400 lignes, tu te perds. Tu cherches une fonction pendant cinq minutes. Tu as peur de casser quelque chose en touchant une ligne au milieu. Tu recopies la meme logique fetch dans trois endroits et tu ne sais plus laquelle est a jour. Les modules existent pour ca : un fichier, une responsabilite, des imports et exports explicites.

En JavaScript moderne, tu ecris export pour sortir une fonction ou une valeur d'un fichier, et import pour la recuperer ailleurs. Dans le HTML, tu charges ton point d'entree avec type="module". Le navigateur sait alors que les fichiers peuvent s'importer entre eux avec des chemins relatifs.

```
<script type="module" src="main.js"></script>
```

Chez DanielCraft, on conseille de decouper des que tu recopies la meme fonction deux fois, ou des que tu ne trouves plus rien sans Ctrl+F. Lea decoupe en api.js, affichage.js et main.js des la deuxieme feature. Max met tout dans un fichier au debut, puis migre quand ca devient confus. Sam montre les deux phases pour que ses eleves comprennent le pourquoi.

Un exemple simple

Fichier api.js :

```
export async function chargerListe(url) {
  const reponse = await fetch(url);
  if (!reponse.ok) throw new Error("Chargement impossible");
  return reponse.json();
}
```

Fichier main.js :

```
import { chargerListe } from "./api.js";

const produits = await chargerListe("/api/produits.json");
console.log(produits);
```

Tu vois l'idée. `main.js` orchestre. `api.js` parle au réseau. Plus tard, tu peux ajouter `afficher.js` pour le DOM. Chaque fichier reste court et se présente en une phrase : "Moi, je charge les données." "Moi, je dessine la liste." "Moi, je branche les boutons." Imagine une cuisine. Un poste pour les entrées, un pour les plats, un pour le service. Chacun a son rôle. Si le serveur commence à cuisiner, le chaos arrive. Si `api.js` manipule le DOM et que `afficher.js` appelle `fetch`, pareil. Un rôle par fichier.

Pièges à connaître

Les modules fonctionnent mieux avec un petit serveur local (pas toujours en ouvrant le fichier en `file://`). Si tu testes avec Live Server, VS Code, ou `python -m http.server`, tu es bien. Les chemins relatifs comptent : `./api.js` veut dire "dans le même dossier". Oublie le `./` et certains navigateurs se plaignent. N'exporte que ce qui est utile. Tout exporter "au cas où" reforge le désordre sous une autre forme.

Tu peux aussi faire export default pour une seule export principale, mais named exports (export function ...) sont souvent plus clairs quand tu as plusieurs fonctions.



Exemple : `main`, `api`, `afficher` - chacun son rôle.

Petite histoire

Lea a hérité d'un projet client : un seul fichier de 900 lignes. Elle a extrait `chargerProduits` dans `api.js` et `afficherProduits` dans `afficher.js` en une matinée. Le client n'a rien vu changer côté utilisateur. Mais Lea a retrouvé sa santé mentale. Max, lui, a attendu trop longtemps et a dû tout refaire. Morale : découpe tout, même pour un exercice.

Erreur classique

Oublier `type="module"` dans le script HTML : les imports ne marchent pas. Tester en `file://` et conclure que "les modules sont cassés" alors qu'il faut un serveur local. Mettre `fetch` et manipulation DOM dans le même fichier "parce que c'est plus rapide" et regretter deux semaines plus tard.

En vrai

Sur un vrai projet, tu n'as pas besoin de vingt fichiers le premier jour. Trois fichiers clairs battent un monstre de 800 lignes. Ouvre un vieux script unique. Identifie deux fonctions exportables. Écris leurs noms d'export. Tu n'as pas besoin de tout migrer maintenant. Juste sentir la découpe.

A toi

Prends un vieux script unique (ou celui du mini-projet). Identifie deux fonctions que tu pourrais sortir dans `api.js` et `afficher.js`. Ecris mentalement leurs signatures `export`. Puis fais la migration réelle si tu as le temps. L'atelier modules au chapitre 16 te guidera pas à pas.

import vs export : le vocabulaire

`export function maFonction() {}` rend la fonction disponible à l'extérieur du fichier. `import { maFonction } from './fichier.js'` la récupère. Les accolades importent un nom précis. Les modules ES6 utilisent des chemins relatifs avec extension `.js` explicite (bonne pratique navigateur). Si tu vois `import truc from "lodash"` sans chemin, c'est souvent un bundler (hors scope ici).

Cycle de vie module

Chaque module n'est évalué qu'une fois par page. Les imports sont "live bindings" pour les variables exportées, mais pour commencer retiens surtout : une fonction exportée est une fonction partagée, pas recopiée. Pratique pour garder une seule fonction `chargerListe`.

Zoom DanielCraft

Quand Lea livre un petit site vitrine avec `fetch`, elle livre souvent `index.html`, `styles.css`, `main.js`, `api.js`, `afficher.js`. Le client peut lire la structure. Sam note les élèves sur la clarté des exports. Max n'a pas besoin de tout comprendre, mais il sait où modifier le texte d'erreur (souvent `afficher.js` ou `main.js`). C'est déjà une victoire organisationnelle.

Chapitre 10 - Organiser un petit projet



Un role par fichier pour maintenir sans peur.

Un projet web, ce n'est pas seulement "du code qui marche une fois". C'est aussi un rangement que tu comprends dans une semaine, quand tu auras oublié les détails, quand un client demandera une modification, ou quand tu reviendras après des vacances. Voici une structure simple qui marche pour beaucoup de mini-apps JavaScript front. Pas de framework, pas de bundler obligatoire. Juste des fichiers avec des rôles clairs.

Chez DanielCraft, on voit trop de débutants coller 300 lignes entre deux balises script dans index.html. Ça marche pour dix lignes. Au-delà, c'est une nasse. Passe tout aux fichiers séparés, même pour un exercice. Lea structure ainsi presque tous ses petits projets clients avant d'envisager React ou autre.

La structure de base

```
mon-projet/  
  index.html  
  styles.css  
  main.js  
  api.js  
  afficher.js  
  data/ (optionnel)
```

index.html : la page, la structure, peu de logique. styles.css : l'habillage. main.js : le chef d'orchestre (écoute les clics, lance les chargements, gère les erreurs). api.js : fetch et parsing réseau. afficher.js : créer des éléments DOM à partir des données. data/ : fichiers JSON locaux pour les exercices.

Responsabilités

Si api.js commence à manipuler le DOM, tu mélanges les rôles. Si afficher.js appelle fetch, pareil. Garde une règle : chaque fichier a une phrase pour se présenter. "Moi, je charge les données." "Moi, je dessine la liste." "Moi, je branche les boutons." Si tu ne trouves pas la phrase, le fichier n'est peut-être pas nécessaire, ou tu dois le renommer. Ton projet est une petite équipe. main.js est le chef : il donne les ordres, ne fait pas tout lui-même. api.js est le messenger : il va chercher les infos dehors. afficher.js est le graphiste : il met en forme à l'écran. index.html est la scène. styles.css est le décor. Chacun son métier.

Noms clairs

faireTruc.js n'aide personne six mois plus tard. chargerProduits, afficherErreur, viderListe : on comprend. Les noms longs et explicites battent les noms courts et mystérieux. Max a renommé tout.js en main.js et data.js en api.js un dimanche. Il a gagné une heure la semaine suivante.

Une seule source de vérité

Évite d'avoir le même tableau de produits copié dans trois endroits. Charge une fois, stocke dans une variable (ou un petit état simple), puis affiche. Si tu modifies, tu modifies à un seul endroit. Sam explique ça à ses élèves avec l'image du tableau au tableau : une seule copie officielle, pas trois versions divergentes.

Petite histoire

Lea a repris le site d'un artisan (pas Max, un autre client). Tout était dans index.html : HTML, CSS inline, JS mélange. En une journée, elle a extrait CSS et JS, découpe en trois modules. Le site faisait la même chose pour l'utilisateur. Mais Lea pouvait enfin ajouter une feature sans peur. L'organisation, c'est de l'argent gagné en maintenance.

Erreur classique

Créer vingt fichiers vides "pour faire pro" avant d'avoir vingt lignes qui marchent. Commence petit : trois fichiers suffisent. Autre piège : copier-coller le même fetch dans main.js et api.js "temporairement" et oublier de supprimer le doublon. Ou mélanger logique métier et affichage dans la même fonction de 80 lignes.

En vrai

Beaucoup de projets open source ou tutos mélangent tout au début. C'est normal. L'étape suivante, c'est ranger. Dessine sur papier les 4 fichiers de ton prochain mini-projet et une phrase de rôle pour chacun. Si une phrase est floue, simplifie.

A toi

Dessine sur papier (ou dans un commentaire en tête de main.js) les fichiers de ton prochain mini-projet et le rôle de chacun. Si tu ne trouves pas la phrase en une ligne, le découpage n'est pas encore clair. Ajuste avant de coder. Ce plan de dix minutes évite des heures de refactor plus tard.

Versioning et sauvegarde

Même pour un petit projet, un dossier Git ou une copie datée sauve du désespoir. Lea commit après chaque feature qui marche ("formulaire ok", "fetch ok"). Max zippe son dossier le dimanche. Sam demande un export GitHub aux élèves motivés. L'organisation, ce n'est pas que les noms de fichiers : c'est aussi pouvoir revenir en arrière quand tu casses fetch en "corrigeant" autre chose.

Evolutive sans sur-ingénierie

Tu n'as pas besoin d'architecture hexagonale pour afficher dix produits. Tu as besoin de ne pas peindre dans un coin. Si tu sens que main.js dépasse cent cinquante lignes, découpe. Si api.js contient des sélecteurs DOM, déplace. Si tu prévois d'ajouter une recherche plus tard, garde les données en mémoire après le premier fetch (atelier 15 bonus). Anticiper un peu, pas tout abstraire d'avance.

Checklist avant de dire "c'est fini"

index.html minimal et lisible. CSS separe. fetch avec ok et catch. Formulaire avec preventDefault si present. Modules si plus de deux fichiers JS. Test en local avec serveur, pas file://. Message chargement et message erreur visibles. Console sans erreur rouge non geree. Si tu coches ca sur un mini-projet DanielCraft, tu es deja au-dessus de beaucoup de tutos abandonnes a mi-chemin.

Chapitre 11 - Deboguer mieux



Reproduire, lire la console, isoler une ligne.

Quand ça casse, le réflexe humain c'est "je recopie tout depuis Stack Overflow". Le réflexe utile, c'est : lire le message, trouver la ligne, formuler une hypothèse, vérifier une chose à la fois. Deboguer, ce n'est pas un échec. C'est une partie normale du métier. Les développeurs expérimentés passent une bonne part de leur temps à lire des erreurs. Chez DanielCraft, on le dit franchement : savoir déboguer compte autant que savoir écrire du code neuf.

Lea débogue avec la console et les breakpoints chaque jour. Max a appris à ne plus paniquer devant le rouge. Sam apprend à ses élèves que l'erreur est un indice, pas une punition. Ce chapitre te donne les outils de base du navigateur, sans IDE complexe.

La console, ton amie

`console.log` n'est pas honteux. C'est un projecteur. Affiche la valeur juste avant l'endroit douteux. Affiche `reponse.status`. Affiche `typeof data`. Affiche `data.length`. Souvent, le bug saute aux yeux : `undefined` là où tu attendais un tableau, clé JSON mal orthographiée, sélecteur qui renvoie `null`.

Pense aussi à `console.error` pour les vrais problèmes (dans un `catch` par exemple), et à retirer les logs inutiles avant de montrer ton travail à un client. Un mur de "test 1", "test 2" dans la console n'aide personne en prod.

Lire une stack trace

Quand le navigateur affiche une erreur rouge, il donne souvent un fichier et un numéro de ligne. Clique dessus dans les DevTools. Regarde la pile : "appelle depuis telle fonction, elle-même depuis telle autre". Tu remontes le fil. Si le message dit `Cannot read properties of null`, tu as probablement sélectionné un élément qui n'existe pas encore (DOM pas prêt), ou un mauvais sélecteur (`#liste` vs `.liste`).

Points d'arrêt (breakpoints)

Dans les outils développeur (F12), onglet Sources, tu peux cliquer à gauche d'une ligne pour poser un breakpoint. La page s'arrête là quand le code passe. Tu inspectes les variables en direct. Tu avances pas à pas (step over, step into). C'est plus précis qu'un mur de logs quand le bug est subtil ou dépend de l'ordre d'exécution.

Hypotheses courtes

Ecris mentalement : "Je pense que data n'est pas un tableau." Puis verifie avec `Array.isArray(data)`. Une hypothese a la fois. Sinon tu changes trois choses en meme temps et tu ne sais plus ce qui a marche. Lea colle parfois un post-it : "Hypothese 1 : mauvaise URL." Elle teste. Puis hypothese 2.

Reseau et DOM

Onglet Network : ta requete fetch est-elle partie ? Quel status HTTP ? Quel corps de reponse ? Parfois le bug n'est pas dans ton JS mais dans ce que le serveur renvoie. Onglet Elements : ton `#liste` existe-t-il au moment ou tu essaies de le remplir ? Si ton script est en head sans defer, le DOM n'est peut-etre pas pret.

Petite histoire

Max a passe une heure a "reparer fetch" alors que son id etait liste-produits dans le HTML et listeProduits dans le JS. `querySelector` renvoyait null. Un `console.log(liste)` avant la boucle aurait tout debloque en deux minutes. Depuis, il loggue les variables suspectes avant de supposer que le reseau est en cause.

Erreur classique

Changer dix lignes a la fois "pour voir". Ajouter des catch vides qui avalent l'erreur. Ignorer le message complet et ne lire que la premiere ligne. Copier une solution sans comprendre pourquoi ca marchait. Ou ne jamais ouvrir les DevTools et rester dans le flou.

En vrai

Prends un vieux bug (ou invente-en un : mauvais id HTML, await oublie). Force l'erreur. Lis le message complet a voix haute. Relie-le a une ligne precise dans ton code. Pose un breakpoint ou un log strategique. C'est l'entrainement qui transforme la panique en routine.

A toi

Prends un code qui marche (mini-projet ou atelier). Casse-le volontairement (mauvais selecteur, oubli de await). Debogue sans regarder la solution. Note en trois lignes ce qui t'a aide : console, Network, breakpoint ? Garde cette mini-fiche pour la prochaine fois.

Routine de debug en cinq minutes

1) Reproduis le bug une fois, note l'action exacte. 2) Lis l'erreur complete dans la console. 3) Clique la ligne, regarde les variables. 4) Formule une hypothese unique. 5) Teste un seul changement. Lea suit cette routine meme sous pression client. Ca evite le "j'ai tout change et je ne sais plus".

Erreurs fetch frequentes a reconnaitre

Failed to fetch : souvent reseau, CORS, ou mauvaise URL. Unexpected token < in JSON : tu parses du HTML (page d'erreur) en JSON. Cannot read properties of undefined : souvent data mal structuree ou await oublie. 404 sans crash fetch : oublie de verifier ok. Garde cette liste dans un coin de ton bureau virtuel.

Outils complementaires

L'onglet Network montre le timing : ta lenteur vient-elle du reseau ou de ta boucle DOM ? L'onglet Application montre localStorage si tu stockes du JSON. Preserve log en console garde les messages apres rechargement : utile quand le bug arrive au submit. Sam fait une demo live de ces onglets : dix minutes qui valent des heures de guess.

Chapitre 12 - Mini-projet : liste depuis une API



Mini-projet : recuperer des donnees et les montrer proprement.

On assemble tout ce que tu as vu. Objectif : une page qui charge une liste (produits, citations, posts fictifs) depuis une URL JSON, affiche les titres ou noms, et gere le cas d'erreur proprement. Tu peux utiliser une API publique de demo, ou un fichier produits.json servi en local. L'important n'est pas la source exacte. C'est le parcours complet : evenement, fetch, verification, affichage, echec gracieux.

Chez DanielCraft, ce mini-projet est le passage oblige entre "j'ai lu les chapitres" et "je sais livrer une petite feature". Lea le fait systematiquement avec les juniors. Max l'a utilise pour sa liste de prestations. Sam l'adapte en exercice note. Si tu reussis les deux chemins (succes et erreur), tu as le socle. Sans le chemin erreur, tu as seulement une demo qui sourit par beau temps.

Ce que ce n'est pas

Ce mini-projet, ce n'est pas un framework. Ce n'est pas non plus "tout decouper en micro-modules des la premiere heure". Ce n'est pas une excuse pour afficher une stack trace au visiteur. Et ce n'est surtout pas "ca marche chez moi en file://" : sers le dossier en local, comme en vrai.

Un bouton "Charger". Un clic. Un message "Chargement...". Puis soit une liste remplie, soit "Impossible de charger. Reessaie." Pas de page blanche. Pas de spinner infini. Pas de stack trace affichee au visiteur. C'est ca, une appli minimale mais pro. Lea visualise le parcours comme un entonnoir : clic, attente, succes ou message humain. Max aussi. Sam force les eleves a dessiner les deux sorties avant de coder.

★ A RETENIR

Deux chemins obligatoires : liste OK, et erreur claire. Sans l'erreur, ce n'est pas un outil.

Parcours en etapes

1. HTML : un titre, un bouton "Charger", une zone #liste, une zone #message.
2. Au clic, tu appelles une fonction async qui fait fetch.
3. Tu verifies reponse.ok.
4. Tu parses le JSON avec await reponse.json().
5. Tu vides #liste, puis tu ajoutes un element par item.
6. Si ca rate, tu ecris un message humain dans #message.

Squelette de depart

```
async function charger() {
  const message = document.querySelector("#message");
  const liste = document.querySelector("#liste");
  message.textContent = "Chargement...";
  try {
    const reponse = await fetch("./produits.json");
    if (!reponse.ok) throw new Error("HTTP " + reponse.status);
    const data = await reponse.json();
    liste.innerHTML = "";
    for (const item of data) {
      const li = document.createElement("li");
      li.textContent = item.nom;
      liste.appendChild(li);
    }
    message.textContent = data.length + " elements";
  } catch (e) {
    message.textContent = "Impossible de charger. Reessaie.";
    console.error(e);
  }
}
```

Adapte item.nom selon ta source (titre, texte, name). L'ossature reste identique. Lea change souvent la cle. Max aussi. L'important, c'est le schema : ok, json, vider, remplir, catch.

Variante meteo (idee)

Meme structure : bouton, chargement, affichage temperature ou description, erreur claire. L'API change. Le schema mental reste. Max a fait sa widget meteo locale en une soiree avec ce squelette. Lea ajoute parfois un second bouton "Actualiser" qui rappelle la meme fonction. Sam propose une variante "citations" pour varier les plaisirs sans changer le muscle.

Qualite minimale attendue

Pas de page blanche silencieuse. Pas d'erreur technique crachee a l'utilisateur (TypeError en gros sur la page). Un etat "Chargement..." visible pendant l'attente. Des noms de variables lisibles (liste, message, reponse, pas x et tmp). Un console.error dans le catch pour toi. Si tu decoupes en modules (api.js + afficher.js), c'est un bonus, pas une obligation pour valider le mini-projet.

Chez DanielCraft, on ajoute souvent un critere invisible : un ami comprend-il la page sans lire le code ? Bouton clair, message clair, liste claire. Si oui, tu as livre une feature, pas un exercice.

Petite histoire

Sam a fait faire ce mini-projet a ses eleves. La moitie a oublie response.ok et affichait "0 elements" sur une 404. L'autre moitie a casse l'URL volontairement et a vu le message d'erreur. Devine qui a vraiment compris fetch ? Ceux qui ont teste l'echec.

Lea a montre sa page a un client non technicien. Le client a clique, vu "Chargement...", puis la liste. Il a dit "c'est clair". Elle n'a pas parle de async/await. Elle a livre une sensation. Chez DanielCraft, on valide ca.

Erreur classique

Oublier de servir le dossier avec un serveur local : fetch sur file:// echoue souvent. Ne pas tester l'URL cassee : tu crois que tout va bien alors que tu n'as jamais vu le catch en action. Afficher les donnees sans vider la liste avant : les items s'accumulent a chaque clic. Autre piege : laisser le message "Chargement..." apres succes. Efface ou remplace.

⚠ ATTENTION

Tester seulement le beau chemin, c'est se mentir. Casse l'URL une fois. Regarde le catch. Puis reparer.

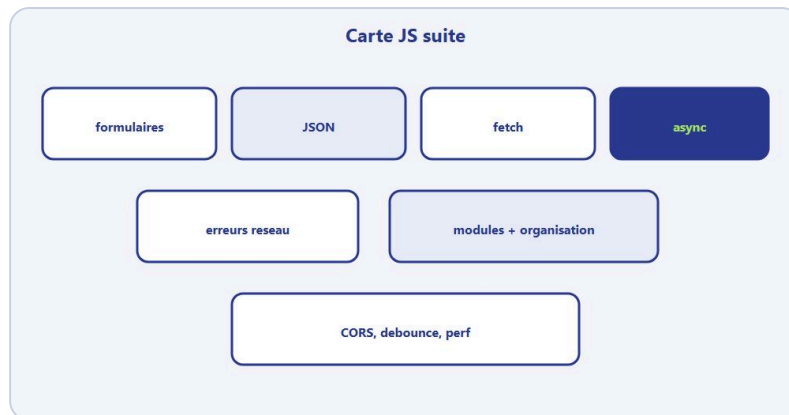
En vrai

Code cette page en local. Casse volontairement l'URL. Verifie que ton message d'erreur apparait. Remets la bonne URL. Verifie la liste. Si ces deux chemins marchent, le mini-projet est reussi. Montre-le a quelqu'un : peut-il comprendre ce qui se passe sans lire le code ?

A toi

Implemente la page complete. Puis decoupe optionnellement en api.js et afficher.js si tu te sens pret (sinon, l'atelier 16 le fera). Ecris en une phrase ce que tu as appris en forcant une erreur reseau. Garde cette phrase : c'est ta preuve que tu as depasse la lecture passive.

Chapitre 13 - A retenir (carte mentale)



Carte : formulaires, JSON, fetch, async, modules, perf.

Si tu ne devais garder qu'une page de ce livre, ce serait celle-ci. JavaScript cote navigateur, au-dela des bases, c'est surtout : faire parler ta page avec le monde exterieur sans mentir a l'utilisateur quand ca charge ou quand ca casse. Tu valides, tu envoies ou tu recois du JSON, tu attends proprement, tu decoupes ton code, tu debogues avec methode.

Chez DanielCraft, on dit souvent : les gestes battent les listes de methodes. Relis cette carte avant un atelier, avant un ticket front, avant de paniquer sur une 404. Lea la garde ouverte. Max la recopie a sa main. Sam la projette cinq minutes en debut de seance. Trois usages, une meme colonne vertebrale.

Les idees solides

Formulaires : preventDefault, trim, validation claire, messages humains, focus sur le champ en erreur. JSON : parse pour lire du texte reçu, stringify pour envoyer. fetch GET : demander une ressource. fetch POST : envoyer un body JSON avec method, Content-Type et stringify. Promesses : then pour le succes, catch pour l'echec, return dans la chaine. async/await : meme flux, ecriture lineaire, try/catch autour des await. Erreurs : response.ok obligatoire, message simple a l'ecran, console.error pour le detail. Modules : export/import, type="module", un role par fichier. Organisation : main orchestre, api reseau, afficher DOM. Debug : une hypothese a la fois, console et Network. CORS : regle navigateur entre origines, pas un bug JS magique. Debounce : attendre un silence avant d'agir. Perf : pas de fetch spam, pas de DOM inutile.

★ A RETENIR

"Pas d'exception ne veut pas dire succes ; undefined n'est pas une liste ; [object Object] n'est pas du JSON."

La boucle DanielCraft

1) Evenement utilisateur. 2) Validation locale. 3) Etat "chargement" visible. 4) fetch avec await. 5) Verifier ok. 6) json(). 7) Afficher ou envoyer. 8) catch avec message humain. 9) Decouper en fichiers des que ca grossit.

C'est la meme boucle pour Lea sur un formulaire contact, pour Max sur sa meteo locale, pour Sam sur une liste de citations. Les donnees changent. Le geste reste.

Ce que tu peux oublier

Le nom exact de chaque API publique de demo. La syntaxe parfaite du premier coup. La peur du rouge dans la console. Garde les gestes. Change d'outil ou d'API si besoin. Les gestes restent.

Petite checklist de poche

Ai-je preventDefault sur le submit ? Ai-je stringify avant POST ? Ai-je verifie ok ? Ai-je un message si ca rate ? Ai-je un "Chargement..." ? Mon fichier depasse-t-il 200 lignes sans decoupe ? Si oui partout sauf le dernier point, tu es solide pour un petit projet reel.

Personnages, meme logique

Lea valide puis POST. Max affiche la meteo avec catch. Sam charge des listes pour ses quiz. Trois metiers, un schema : lire, verifier, agir, expliquer l'echec.

Phrase talisman

"Pas d'exception ne veut pas dire succes ; undefined n'est pas une liste ; [object Object] n'est pas du JSON." Garde-la. Elle resume les trois bugs les plus frequents de ce livre.

Petite histoire

Lea a voulu tout retenir d'un coup. Elle a bloque. Elle a affiche cette carte au-dessus de l'ecran et a refait le squelette async + fetch a voix haute. Max a saute response.ok "parce que ca marchait en demo" : page blanche chez le client. Sam a force une URL cassee en classe : les eleves ont enfin vu le catch. Trois lecons, une carte. Chez DanielCraft, on prefere une carte tenue a une memoire heroique.

Erreur classique

Croire que "j'ai lu" egal "je sais assembler". Sans livrable (mini-projet, atelier), le cerveau classe ca comme "vu", pas comme "su". Autre piege : collectionner les chapitres sans jamais casser volontairement une URL. Tu ne verras jamais ton vrai catch.

⚠ ATTENTION

Sans tester l'echec, tu n'as qu'une moitie de competence. Casse l'URL. Verifie le message. Remets.

En vrai

Sans regarder le livre, ecris de memoire le squelette async + fetch + ok + json + catch. Chronometre cinq minutes. Compare a tes notes. Les trous montrent ce qu'il faut relire avant les ateliers. Puis coche la checklist de poche sur ton dernier script.

A toi

Recopie cette carte sur papier en 12 lignes max, avec TES mots. Puis ecris de memoire le squelette async + fetch + ok + json + catch. Compare a tes notes. Les trous montrent ce qu'il faut relire avant les ateliers.

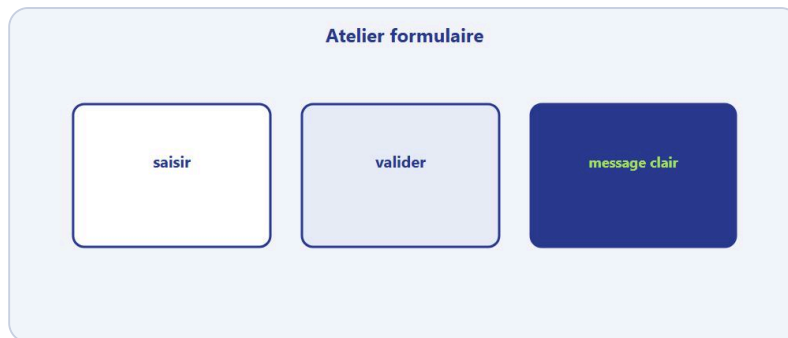
Note de rythme

Prends le temps. Un atelier fait a fond vaut mieux que trois ateliers survoles. Si tu es presse, fais la moitie aujourd'hui et l'autre demain - mais ecris le livrable. Sans livrable, le cerveau classe ca comme "lu", pas comme "su". DanielCraft forme des gens qui livrent, meme petit.

Pour aller plus loin sans te perdre

Tu n'as pas a tout maitriser d'un coup. Reviens a ce chapitre quand tu butes sur un cas reel (CORS, POST, module qui ne charge pas). Relis l'erreur classique du chapitre concerne. Refais le "A toi". Chez DanielCraft, on prefere trois relectures actives a une lecture passive de vingt pages. Si un paragraphe te semble encore flou, reformule-le a voix haute avec ton propre exemple (meteo, contact, produits). Des que ca passe a l'oral, c'est que c'est entre.

Chapitre 14 - Atelier : formulaire robuste



Saisir, valider, message clair.

Objectif : construire un formulaire contact (nom, email, message) qui refuse l'envoi si un champ manque ou est invalide, et qui explique clairement quoi corriger. Duree : 30 a 45 minutes. Matériel : editeur de code, navigateur, eventuellement Live Server ou python -m http.server.

Lea fait faire cet atelier a chaque stagiaire avant de le laisser toucher a un vrai formulaire client. Max l'a refait pour son site apres avoir reçu des mails vides. Sam le note sur dix en fin de seance. Ce n'est pas glamour. C'est fondamental. Chez DanielCraft, un formulaire qui laisse partir du vide, c'est un devis qui part trop tot : tu livres du bruit.

Tu vas sentir la difference entre "ca marche en demo" et "ca tient quand quelqu'un tape n'importe quoi". Le navigateur a ses validations HTML. Toi, tu restes le chef pour les messages clairs, le focus, et le preventDefault qui empeche la page de recharger au moment ou tu voulais juste lire les champs.

Exercice 1 - Structure HTML (10 min)

Cree index.html avec un formulaire id="contact", trois champs (nom, email, message), un bouton submit, une zone #erreurs pour les messages. Utilise des label for lies aux id des inputs. Ajoute type="email" sur l'email si tu veux, mais rappelle-toi : JS reste le chef pour les messages clairs. Sans label, un champ "marche" encore - mais Sam refuse la note : l'accessibilite commence ici.

Exercice 2 - Ecouter submit (10 min)

Selectionne le formulaire. Ecoute submit. Appelle preventDefault() tout de suite. Lis les valeurs avec .value.trim(). Loggue l'objet { nom, email, message } en console pour verifier que la lecture marche avant de valider. Si tu vois encore un rechargement de page, preventDefault n'est pas au bon endroit - ou tu ecoutes le mauvais evenement.

Exercice 3 - Validation par liste d'erreurs (15 min)

Construis un tableau erreurs = []. Si nom vide : pousse "Indique ton nom." Si email sans @ : pousse "Email incomplete." Si message moins de 10 caracteres : pousse "Message trop court." Affiche toutes les erreurs dans #erreurs (join avec retour ligne ou ul). Si aucune erreur, affiche "Formulaire pret (simulation d'envoi)." et loggue l'objet propre en console. Lea prefere une liste d'erreurs a une seule alerte : l'utilisateur corrige tout d'un coup.

Exercice 4 - Focus et reset (10 min)

Après affichage d'une erreur, mets le focus sur le premier champ concerne. Quand tout est valide, simule un envoi : message de succes, puis `form.reset()` apres deux secondes (`setTimeout`). Ca prepare le vrai POST du chapitre 8. Max a adore ce detail : apres envoi, la page est prete pour le prochain client sans F5.

Livrable

Un dossier atelier-formulaire/ avec `index.html` et `app.js` (ou script inline si tu preferes). Capture ou note : une capture d'ecran avec erreurs affichees, une avec succes. Sans livrable, le cerveau classe ca comme "lu".

Criteres de reussite

Sans JS, le navigateur ne doit pas naviguer a cause du submit (`preventDefault` marche). Les messages sont en francais simple, pas des `alert()`. Tu peux envoyer seulement quand tout est valide. Tu logs l'objet `{ nom, email, message }` une fois valide. Les labels sont presents.

Petite histoire

Max a reçu trois mails "contact" vides en une semaine. Il croyait que "type=email" suffisait. Lea lui a fait refaire cet atelier : trim, liste d'erreurs, focus. Plus de mails fantomes. Sam a chronometre ses eleves : ceux qui testent volontairement le champ vide comprennent plus vite que ceux qui ne cliquent que le chemin heureux.

Erreur classique

Ne te contente pas de `alert()`. Les alertes agacent et bloquent la page. Un message dans `#erreurs` est plus pro, meme pour un exercice. Ne valide qu'au clic bouton sans écouter submit : Entree dans un champ ne declenchera pas ta logique. Autre piege : oublier trim et accepter " " comme un vrai nom.

⚠ ATTENTION

Un espace seul n'est pas un nom. `trim()` avant de juger. Sinon tu valides du vide deguise.

Variante avancee

Desactive le bouton pendant une fausse attente d'une seconde (`setTimeout`), puis reactive-le. Ajoute une validation "email doit contenir un point apres le @". Documente la regle en commentaire.

En vrai

Remplis le formulaire correctement, puis vide le nom, puis mets un email sans @, puis un message de trois lettres. A chaque fois, verifie le message et le focus. Si les trois chemins malheureux sont clairs, l'atelier tient. Montre la page a quelqu'un : comprend-il quoi corriger sans que tu parles ?

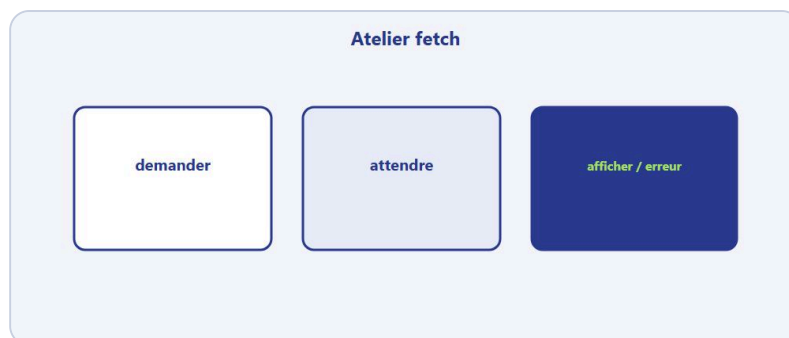
Note de rythme

Prends le temps. Un atelier fait a fond vaut mieux que trois ateliers survoles. Si tu es presse, fais la moitie aujourd'hui et l'autre demain - mais ecris le livrable. Sans livrable, le cerveau classe ca comme "lu", pas comme "su". DanielCraft forme des gens qui livrent, meme petit.

A toi

Termine le livrable. Puis écris en trois lignes ce qui était le plus dur (preventDefault, trim, affichage erreurs ?). Garde ce retour pour le prochain formulaire réel.

Chapitre 15 - Atelier : fetch et afficher



Demander, attendre, afficher ou erreur.

Objectif : charger un fichier JSON local et remplir une liste HTML avec gestion complete du chargement et des erreurs. Duree : 30 a 45 minutes. Matériel : editeur, serveur local (Live Server ou python -m http.server).

Sam utilise cet atelier avant le chapitre modules : d'abord tout dans un fichier, puis decoupe. Lea verifie que response.ok et le message d'erreur sont presents. Max l'a fait avec des citations au lieu de produits. L'important : le parcours fetch complet. Chez DanielCraft, on ne valide pas "ca affiche quelque chose". On valide les deux chemins : succes et echec.

Tu vas sentir pourquoi file:// te ment souvent. fetch aime un vrai serveur local, meme minuscule. Si tu ouvres le HTML en double-clic et que "ca marche pas", ce n'est pas forcément ton code - c'est le protocole. Note ca. Tu gagneras une heure la prochaine fois.

Preparation

Cree un fichier citations.json a cote de ta page :

```
[
  { "texte": "Petit a petit, l'oiseau fait son nid.", "auteur": "Proverbe" },
  { "texte": "On apprend en faisant.", "auteur": "DanielCraft" },
  { "texte": "Le code clair se lit comme une histoire.", "auteur": "Atelier" }
]
```

Sers le dossier avec un petit serveur local. Ouvrir index.html en file:// fera souvent echouer fetch : c'est normal, pas un bug de ton code.

Exercice 1 - HTML et bouton (5 min)

Page avec titre, bouton "Charger les citations", ul#liste, p#message. Style minimal optionnel. Lea ajoute parfois un petit CSS pour que ca ressemble a une vraie mini-app - pas obligatoire pour valider.

Exercice 2 - fetch async (15 min)

Au clic, message.textContent = "Chargement...". fetch("./citations.json"). Verifie reponse.ok. await reponse.json(). Vide la liste. Pour chaque item, cree un li avec texte et auteur (ex. "texte - auteur"). message = N citations chargees. Si tu oublies de vider innerHTML, un second clic double la liste : piege classique.

Exercice 3 - Erreurs (10 min)

try/catch autour du tout. URL cassee volontairement (citations.json -> citatons.json) : message humain "Impossible de charger. Reessaie." plus console.error(e). Remets la bonne URL et verifie les trois citations visibles. Sam note surtout cet exercice : sans lui, tu n'as qu'une demi-competence.

Exercice 4 - Bonus recherche locale (15 min optionnel)

Ajoute un input#filtre. A chaque frappe, filtre la liste deja chargee en memoire (pas de nouveau fetch a chaque lettre). Stocke les citations dans une variable let cache = null apres le premier chargement. Le chapitre debounce affinera ce comportement. Max adore ce bonus : ca donne tout de suite l'impression d'un outil.

Livrable

Dossier atelier-fetch/ avec index.html, app.js, citations.json. Deux tests notes : succes (3 items) et echec (URL cassee). Sans les deux, l'atelier n'est pas fini.

Criteres de reussite

Trois citations visibles en cas de succes. URL cassee -> message d'erreur, pas de page blanche. Pas d'exception non geree dans la console (sauf ton log volontaire dans catch). Etat "Chargement..." visible pendant l'attente.

Petite histoire

Sam a fait faire cet atelier a une classe. La moitie a oublie response.ok et affichait "0 citations" sur une 404. L'autre moitie a casse l'URL et a vu le message. Devine qui a vraiment compris fetch ? Ceux qui ont teste l'echec. Lea raconte la meme scene en stage. Chez DanielCraft, c'est devenu un rituel : casse avant de dire "c'est bon".

Erreur classique

Oublier return ou await sur reponse.json(). Oublier de vider innerHTML avant de remplir (double clic = double liste). Tester sans serveur local et conclure que fetch "ne marche pas". Afficher le stack trace a l'utilisateur au lieu d'un message humain.

ATTENTION

file:// n'est pas ton ami pour fetch. Sers le dossier. Sinon tu debogues un fantome.

En vrai

Lance le chemin heureux. Puis renomme le JSON une seconde. Clique. Lis le message. Remets le nom. Clique. Si les deux chemins sont nets, tu as le muscle. Chronometre : en moins de deux minutes, tu dois pouvoir basculer entre succes et echec sans toucher au JS (juste le nom de fichier). Lea chronometre ses juniors. Max aussi, pour lui-meme.

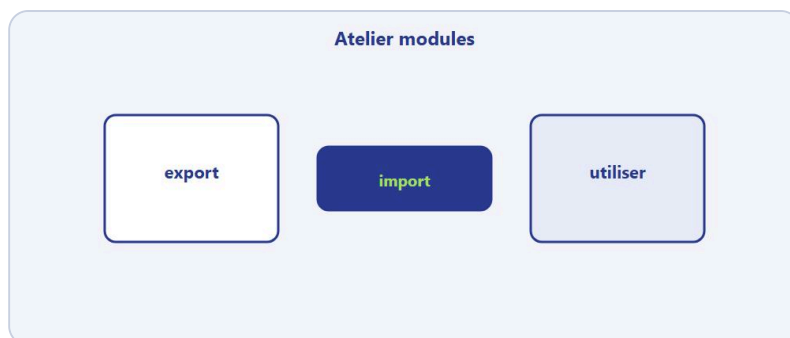
Note de rythme

Prends le temps. Un atelier fait a fond vaut mieux que trois ateliers survoles. DanielCraft forme des gens qui livrent, meme petit. Ecris le livrable, meme minimal.

A toi

Livre l'atelier. Puis reecris la fonction `charger` en `async/await` si tu l'avais faite en `then`. Compare. Garde la version la plus lisible pour l'atelier modules suivant. Bonus : note en une phrase ce que tu as appris en forçant l'erreur.

Chapitre 16 - Atelier : decouper en modules



export, import, utiliser.

Objectif : reprendre l'atelier fetch (ou le mini-projet) et le decouper en trois fichiers avec import/export.

Duree : 30 a 45 minutes. Matériel : atelier-fetch fonctionnel, serveur local.

Lea dit : "Si tu ne peux pas expliquer le role de chaque fichier en une phrase, tu as decoupe trop tot ou pas assez." Cet atelier force la clarte. Max a gagne en lisibilite. Sam note la capacite a expliquer a voix haute. Chez DanielCraft, decouper n'est pas "faire joli". C'est pouvoir toucher le reseau sans casser l'affichage, et toucher l'affichage sans casser le reseau.

Tu pars d'un fichier unique qui marche. Tu le coupes. Tu verifies que ca marche encore - succes et erreur. Si ca casse, tu as decoupe trop vite ou oublie `type="module"`. Le refactor qui "semble propre" mais ne tourne plus n'est pas un refactor. C'est une illusion.

Les fichiers cibles

`api.js` porte le reseau : tu y exportes une fonction async `chargerCitations` qui fait le fetch, verifie ok, et renvoie le JSON. `afficher.js` porte le DOM : tu y exportes `afficherCitations(listeEl, data)` qui boucle et cree les li. `main.js` orchestre : il importe les deux, branche le bouton, gere try/catch et les messages. Dans le HTML, un seul script : `type="module"` vers `main.js`.

```
<script type="module" src="main.js"></script>
```

Trois roles, trois phrases. Si une phrase est floue, renomme ou recoupe.

Exercice 1 - Extraire `api.js` (10 min)

Deplace tout le fetch dans `api.js`. Exporte `chargerCitations` sans parametre si l'URL est fixe (`./citations.json`), ou avec `url` en parametre si tu preferes reutiliser. Garde `if (!reponse.ok) throw ...` dedans. Lea insiste : le throw reste dans `api`, pas dans `main`. Ainsi `main` ne connait que "ca a marche" ou "ca a plante".

Exercice 2 - Extraire `afficher.js` (10 min)

Deplace la boucle qui cree les li. Exporte `afficherCitations(liste, data)`. Option : export fonction `afficherMessage(zone, texte)` pour centraliser les textes statut. Max aime cette option : `main` devient encore plus mince.

Exercice 3 - main.js orchestrateur (15 min)

Importe chargerCitations et afficherCitations. Sur clic bouton : message Chargement, try { const data = await chargerCitations(); afficherCitations(liste, data); message succes } catch { message erreur; console.error }. main.js ne doit pas contenir de fetch ni de createElement pour les li (sauf si tu as choisi de garder les messages dans main via afficherMessage). Sam fait expliquer a voix haute avant de noter.

Exercice 4 - Verifier les tailles (5 min)

Aucun fichier ne devrait dépasser environ 40 lignes pour cet exercice. Si un fichier est enorme, tu n'as pas assez decoupe ou tu as laisse du code mort. Supprime les doublons "temporaires".

Livable

Meme dossier qu'avant, restructure. README ou commentaire en tete de main.js : une phrase par fichier. Les deux chemins (succes, URL cassee) doivent encore marcher.

Criteres de reussite

Ca marche comme avant (succes et erreur). Tu peux expliquer a voix haute le role de chaque fichier en une phrase. type="module" present. Serveur local utilise. Pas de fetch dans main.

Bonus

Ajoute export function afficherErreur(zone, texte) dans afficher.js. main.js ne touche plus au textContent des messages directement. Encore plus propre.

Petite histoire

Lea a decoupe trop tot un projet junior : six fichiers pour trente lignes, personne ne savait ou regarder. Max a laisse un monolithe de 400 lignes : impossible de tester le fetch seul. Sam a trouve le juste milieu avec cet atelier : trois fichiers, trois roles, deux chemins verifies. Chez DanielCraft, on dit : decoupe pour comprendre, pas pour impressionner.

Erreur classique

Oublier type="module". Chemins import sans ./ (./api.js). Laisser un doublon de fetch dans main.js "temporairement". Tester en file://. Croire que "ca compile" (il n'y a pas de compile) alors que la console crie Mixed content ou Failed to resolve module.

ATTENTION

Sans type="module", import/export ne demarre meme pas. Verifie le HTML avant de reecrire tout le JS.

En vrai

Apres decoupage, casse encore l'URL. Verifie le message. Remets. Puis explique a quelqu'un (ou a un canard en plastique) le role de chaque fichier en une phrase. Si tu hesites, renomme. La clarte orale predit la clarte du code. Lea le fait a voix haute. Max aussi. Sam l'exige en binome.

Note de rythme

DanielCraft forme des gens qui livrent. Un découpage propre vaut le temps passé. Ne survole pas : fais tourner les deux chemins erreur et succès après refactor.

A toi

Livre la version modulaire. Écris les trois phrases de rôle (api, afficher, main). Si une phrase est floue, renomme ou recoupe encore. Bonus : demande à un ami de lire seulement ces trois phrases et de prédire ou vit le fetch.

Chapitre 17 - CORS, en une page claire



Sans autorisation du serveur, le navigateur bloque.

Tu appelles une API depuis ta page. La console crie quelque chose avec "CORS" ou "blocked by CORS policy". Tu te dis que JavaScript est cassé, que fetch est nul, que tu as fait une faute de frappe. Souvent, non : c'est le navigateur qui applique une règle de sécurité. Comprendre CORS en deux minutes te évite des heures de panique et de mauvaises "solutions" trouvées sur des forums.

Chez DanielCraft, on voit des juniors perdre une journée entière sur CORS sans savoir ce que c'est. Ce chapitre te donne l'essentiel : pas un cours complet de sécurité web, mais assez pour savoir quoi faire quand le message apparaît.

Une page hébergée sur `http://localhost:3000` qui demande des données à `https://autre-site.com` fait une requête "inter-origine". Origine = protocole + domaine + port. Le navigateur demande au serveur distant : "Est-ce que tu autorises cette origine à lire ta réponse ?" Si le serveur ne répond pas oui avec les bons en-têtes CORS (Access-Control-Allow-Origin, etc.), le navigateur bloque la réponse côté page. Ton code fetch échoue. Ce n'est pas toujours toi qui "codes mal". C'est une règle : un site malveillant ne doit pas lire facilement les données privées d'un autre site à ta place sans que le serveur distant l'accepte.

Ce que tu peux faire en apprenant

Utilise des API qui autorisent explicitement le navigateur (demos publiques, APIs avec CORS ouvert pour les tests). Ou passe par ton propre serveur (backend) qui, lui, appelle l'API côté serveur : le navigateur ne voit que ton domaine. Ou travaille avec des fichiers JSON locaux servis par le même origine pendant les exercices de ce livre. Lea développe souvent le front contre un petit backend maison ou des JSON locaux avant de brancher l'API cliente.

Ce qu'il ne faut pas croire

"Désactiver CORS" dans le navigateur via une extension ou un flag n'est pas une solution pro. Ça peut aider cinq minutes pour un test solo, ça ne règle rien pour tes utilisateurs finaux. Ne ship jamais une appli qui dépend de ça. Autre mythe : "CORS c'est only en dev". Non, c'est partout dans le navigateur.

Petite histoire

Max voulait afficher des données d'une API météo gratuite sur son site. Ça marchait quand il ouvrait l'URL directement dans le navigateur. Ça plantait en fetch depuis sa page. Même URL, contexte différent : barre d'adresse vs JavaScript. Lea lui a montré une API alternative avec CORS autorisé, puis un fichier JSON local pour l'exercice. Problème compris, pas contourné en hack.

Sam explique a ses eleves : lire une URL dans la barre, ce n'est pas la meme chose qu'une requete fetch depuis une page. Le navigateur applique des regles differentes.

Lire le message d'erreur

Quand tu verras CORS, lis le message : quelle origine est bloquee ? quelle URL distante ? quelle methode ? Puis demande-toi : est-ce que je suis cense appeler cette API directement depuis le navigateur, ou depuis un serveur intermediaire ? Cette question evite des heures de tentatives inutiles. Parfois la reponse est "utilise leur SDK" ou "inscris-toi pour une cle avec domaine autorise".

Erreur classique

Conclure que fetch est casse et tout reecrire en XMLHttpRequest (meme probleme CORS). Ou copier une extension "Allow CORS" et croire que le projet est fini. Ou blamer le JSON alors que c'est l'origine.

En vrai

Note en une phrase ta definition personnelle de CORS. Si tu peux l'expliquer a un ami sans jargon ("le navigateur verifie si le site distant accepte que MA page lise sa reponse"), c'est gagne. Garde une liste de deux APIs que tu sais utiliser depuis le front sans surprise, et une strategie backend si tu depasses.

A toi

Ecris ta definition CORS en trois lignes max. Puis liste deux options quand une API bloque : API alternative avec CORS OK, ou backend intermediaire. Pas besoin d'implementer le backend ici : comprendre la strategie suffit pour ce livre.

Preflight : une precision utile

Pour certaines requetes (POST avec headers custom, methodes exotiques), le navigateur envoie d'abord une requete OPTIONS "preflight" pour demander la permission. Si le serveur ne repond pas correctement, fetch echoue avant meme ton POST. En exercice avec JSON simple et APIs de demo, tu touches parfois ce cas. Symptome : erreur CORS alors que GET marchait. Solution : lire la doc API ou passer par ton backend.

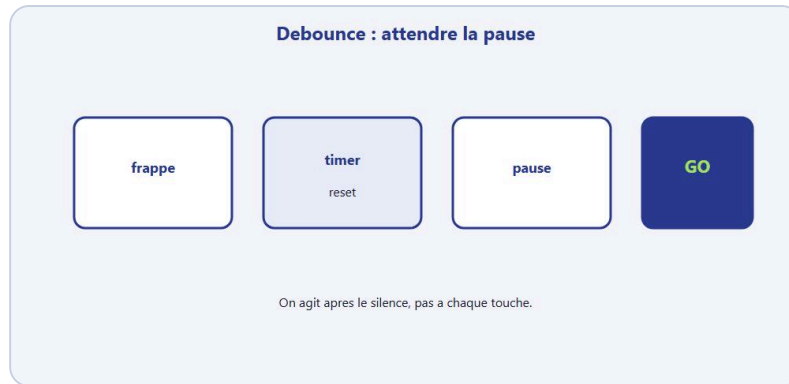
Same-origin en pratique

Meme domaine, meme port, meme protocole : pas de CORS. Fichier servi depuis http://localhost:5500 qui fetch http://localhost:5500/data.json : OK. Page file:// qui fetch n'importe quoi : souvent KO. Page localhost qui fetch api externe : depend des en-tetes du serveur distant. Lea teste toujours en conditions proches de la prod (serveur local, pas file).

En resume DanielCraft

CORS n'est pas un bug de ton code JavaScript. C'est le navigateur qui protege l'utilisateur. Ton job front : utiliser des sources compatibles navigateur, ou deleguer au backend. Ne perds pas une journee a "desactiver CORS". Perds trente minutes a comprendre l'origine du blocage et choisir la bonne strategie.

Chapitre 18 - Debounce : ne pas frapper trop vite



Agir apres la pause, pas a chaque touche.

Imagine une barre de recherche sur un site e-commerce. A chaque lettre tapee, tu lances un fetch vers le serveur. L'utilisateur ecrit "casque" : cinq requetes partent (c, ca, cas, casq, casque). Le serveur souffle. L'affichage clignote entre des resultats partiels. La facture API grimpe. L'experience utilisateur devient nausueuse. Mauvaise idee partout.

Le debounce, c'est attendre un petit silence avant d'agir. Tu tapes, tu tapes, tu pauses 300 ms : la seulement, tu lances la recherche ou le filtre. Une requete utile au lieu de dix inutiles. Lea l'ajoute sur presque toute recherche live. Max l'a decouvert en filtrant sa liste de prestations. Sam le montre apres l'atelier fetch bonus.

Comme un ascenseur avec porte retardee : les gens entrent encore, la porte attend. Personne n'entre plus depuis un moment, la porte se ferme et on part. Chaque nouvelle frappe "reinitialise" le timer. Le voyage n'a lieu qu'apres la pause.

Idee en code

```
function debounce(fn, delai) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delai);
  };
}

const rechercher = debounce((texte) => {
  console.log("Je cherche :", texte);
  // ici : fetch ou filtre local sur cache
}, 300);

input.addEventListener("input", (e) => rechercher(e.target.value));
```

debounce prend une fonction et un delai en ms. Elle renvoie une nouvelle fonction. A chaque appel, le timer precedent est annule et un nouveau démarre. Quand l'utilisateur s'arrete assez longtemps, fn s'execute une fois avec la derniere valeur.

Filtre local vs fetch

Si tu as déjà chargé toutes les citations en mémoire (atelier 15), `debounce` plus `filtre local` suffit souvent : pas de réseau, instantané après la pause. Si tu cherches dans une grosse base distante, `debounce` plus `fetch` évite le spam. Ne confonds pas : `debounce` ne remplace pas le cache ni le `debounce` côté serveur pour des apps énormes, mais pour ce livre c'est largement suffisant.

Quand l'utiliser

Recherche pendant la frappe. Redimensionnement de fenêtre (recalcul layout). Sauvegarde auto d'un brouillon. Partout où un événement se répète trop vite et où tu veux une action "finale" après stabilisation.

Quand ne pas l'utiliser

Un clic sur "Envoyer" : pas besoin, un clic = une action. Un compteur de jeu à chaque frame : ce n'est pas le même outil (`throttle` serait plus adapté, hors scope ici). Une validation de formulaire au submit : non plus.

Petite histoire

Lea a déployé une recherche sans `debounce`. L'API a rate-limité le client en heure de pointe. En ajoutant 300 ms de `debounce` et un `filtre local` sur les résultats déjà chargés, le problème a disparu. Quinze lignes de code, grosse différence.

Erreur classique

Mettre un délai ridiculement court (10 ms) : tu as encore trop de requêtes. Trop long (2000 ms) : l'interface semble molle. 250-400 ms est un bon départ pour une recherche texte. Oublier `clearTimeout` : timers multiples s'empilent. Copier `debounce` sans comprendre : difficile à déboguer.

En vrai

Ajoute un `debounce` sur un champ input qui `console.log` la valeur. Tape vite "bonjour". Vérifie qu'un seul log apparaît après ta pause, pas sept. Ajuste délai au feeling. Si tu vois encore sept logs, ton `clearTimeout` ne tourne pas : relis la fonction.

A toi

Reprends l'atelier `fetch` avec `filtre bonus`. Enveloppe la fonction de filtre dans `debounce(..., 300)`. Teste en tapant vite. Note combien de filtrages réels se déclenchent. Si c'est un par "rafale de frappe", c'est gagné. Bonus : passe à 400 ms et sens la différence de "molle".

Debounce vs throttle (aperçu)

`Debounce` : agir après la pause (recherche). `Throttle` : agir au plus une fois par intervalle fixe (scroll, resize intense). Ce livre reste sur `debounce` car c'est le cas le plus fréquent pour les recherches live. Si un jour tu scroll une carte qui doit se mettre à jour en continu, tu chercheras `throttle`. Ne mélange pas les deux sans raison.

Variante avec fonction nommee

```
function filtrerListe(texte) {  
  // filtre sur cache local  
}  
const filtrerDebounced = debounce(filtrerListe, 300);  
input.addEventListener("input", (e) => filtrerDebounced(e.target.value));
```

Separer `filtrerListe` et `debounce` rend le test plus facile : tu peux appeler `filtrerListe` directement en console sans attendre le delai. Lea garde cette structure pour deboguer plus vite. Max aussi, depuis qu'il a du "sentir" le timer a l'aveugle.

En resume

Debounce protege le reseau, le serveur, et les yeux de l'utilisateur. Ce n'est pas du luxe sur une recherche live. C'est hygiene. Combine au cache local (liste deja chargee), tu obtiens une UX fluide sans infrastructure complexe. DanielCraft recommande ce duo sur tout champ de filtre branche a fetch ou a une grosse liste DOM. Une pause de 300 ms, c'est souvent invisible pour l'humain et precieux pour le serveur.

Chapitre 19 - Petites habitudes de performance



Moins de fetch, moins de DOM, feedback clair.

Pas besoin de devenir expert performance web pour éviter les bêtises coûteuses. Quelques gestes changent déjà beaucoup sur les mini-apps de ce livre : moins de travail inutile, moins de requêtes spam, une attente qui paraît moins longue parce qu'elle est expliquée. Chez DanielCraft, on préfère une appli simple et stable à une appli "optimisée" illisible avec des micro-hacks partout.

Lea optimise après mesure, pas avant. Max évite surtout les erreurs grossières. Sam rappelle : lisibilité d'abord. Ce chapitre est une boussole, pas un manuel d'ingénieur perf.

Moins de travail inutile

Ne recrée pas toute la liste HTML si un seul titre change : mets à jour l'élément concerné. Ne relance pas un fetch à chaque pixel de scroll sans debounce ou sans vraie raison. Ne charge pas une image géante pour une icône de 32 pixels. Si tu as déjà les données en mémoire, filtre localement au lieu de redemander au serveur à chaque lettre (debounce plus cache, chapitre 18). La perf, ce n'est pas "aller plus vite à tout prix". C'est "ne pas gaspiller". Un fetch inutile, c'est du gaspillage. Un DOM détruit et recréé entier pour un changement minuscule, idem. Un spinner infini sans message, c'est du gaspillage de confiance utilisateur.

DOM : batch mental

Quand tu ajoutes 50 éléments, construis-les en mémoire puis ajoute-les (DocumentFragment ou append en une fois). Évite d'alterner lecture et écriture du DOM dans une boucle serrée si tu n'y es pas obligé : chaque toucher au DOM peut déclencher du recalcul layout. Pour 10 li dans un exercice, ce n'est pas critique. Pour 500, ça se voit.

Reseau

Affiche un squelette ou "Chargement..." pour que l'attente soit digeste psychologiquement. Gère les erreurs au lieu de laisser tourner indéfiniment. Évite les fetch en double : si un chargement est en cours, désactive le bouton ou ignore le second clic. Debounce les recherches live. Cache en variable session ce qui ne change pas pendant la visite (liste déjà chargée).

Code lisible d'abord

Optimiser un code confus, c'est peindre une voiture sans moteur. D'abord clair (modules, noms, gestion erreurs). Ensuite mesure si tu sens la lenteur. Ensuite optimise le vrai goulot. Les outils navigateur (onglet Performance, Network avec waterfall) existent pour ca. Ne devine pas au hasard.

Petite histoire

Max avait un bouton "Actualiser" qui relançait fetch a chaque clic nerveux du client. Trois requetes identiques en deux secondes. Lea a ajoute un flag enChargement et desactive le bouton pendant le fetch. Meme donnees, moins de charge, meilleure impression pro.

Erreur classique

Optimiser prematurement : micro-refactor obscur pour gagner 2 ms. Ignorer le reseau : 50 fetch par seconde sur une recherche. Oublier l'UX de l'attente : pas de feedback = "ca marche pas ?". Confondre perf et complexite : plus de code != plus rapide.

En vrai

Relis ton mini-projet ou atelier fetch. Cherche un seul gaspillage concret (fetch en double, pas de debounce sur filtre, liste recreee entierement a chaque filtre mineur). Corrige ce seul point. Mesure au feeling avant/apres. Un fix vaut mieux que dix conseils theoriques. Ouvre Network une fois : compte les requetes. Le chiffre ancre mieux qu'un sentiment.

A toi

Note un gaspillage trouve et comment tu l'as corrige (ou compte le corriger). Si tu n'en trouves aucun, fais relire ton code a quelqu'un ou relis-le demain matin avec des yeux neufs. Il y en a presque toujours un. Bonus : ecris le gaspillage en une phrase dans le README du projet - pour ton futur toi.

Mesurer sans obsession

Onglet Network : combien de requetes au chargement ? Quelle taille du JSON ? Performance : une interaction lente est-elle due au JS ou au layout ? Tu n'as pas besoin de maitriser ces outils a fond. Tu as besoin de regarder une fois pour calibrer ton intuition. Lea ouvre Network quand un client dit "c'est lent" : souvent, c'est six images non compressees, pas le fetch.

Accessibilite et perf

Un bouton desactive pendant chargement evite double clic ET annonce visuellement l'attente. Un message texte vaut mieux qu'un spinner seul pour les lecteurs d'ecran basiques. Ce n'est pas le coeur du chapitre, mais DanielCraft aime les apps qui ne stressent pas l'utilisateur. Perf percue, c'est aussi clarte.

Priorites pour ce niveau

1) Pas de fetch spam (debounce). 2) Pas de DOM reconstruit sans raison. 3) Feedback chargement/erreur. 4) Code decoupe lisible. 5) Optimisations micro seulement si mesure prouve le goulot. Max suit cette liste. Sam la met au tableau. Si tu fais deja 1 a 4, tu es large pour les projets de ce livre. Chez DanielCraft, on repete : ne pas gaspiller bat "optimiser pour le plaisir".

Quiz final

Quiz : verifier sans inventer

Lis. Choisis A / B / C.

Si doute : reouvre le chapitre.

Verifier sans inventer.

Pas de piege. L'idee : verifier que tu peux avancer sur un petit projet reel sans paniquer devant fetch, JSON ou un formulaire. Relis un chapitre si une question te bloque. Lea fait passer ce quiz avant de confier un vrai ticket client front. Sam l'utilise en fin de module. Max l'a refait une fois pour se rassurer. Chez DanielCraft, un quiz sert a reperer les trous, pas a humilier.

Coche sans tricher. Une premiere passe honnete vaut mieux qu'un score maquille. Si tu bloques, marque et continue. Tu reviendras. Le but, c'est la carte des chapitres a rouvrir.

♦ ASTUCE

Fais le quiz, note ton score, rejoue dans une semaine apres un mini projet. Le vrai progres se voit a la deuxieme passe.

Questions

1. 1. Pourquoi appelle-t-on souvent preventDefault() sur un submit de formulaire ?

- A) Pour colorier le bouton
- B) Pour empecher l'envoi / rechargement par default du navigateur
- C) Pour vider le localStorage

1. 2. JSON.stringify sert surtout a :

- A) Transformer un objet JS en texte JSON
- B) Afficher une image
- C) Creer un fichier CSS

1. 3. Que fait await devant un fetch ?

- A) Il ignore la reponse
- B) Il attend la reponse avant de continuer dans la fonction async
- C) Il compile le HTML

1. 4. Pourquoi verifier reponse.ok ?

- A) Parce que fetch ne leve pas toujours d'erreur sur un 404
- B) Parce que JSON est obligatoire

- C) Parce que le DOM l'exige

1. 5. Dans un module, export sert a :

- A) Supprimer une fonction
- B) Rendre une fonction (ou valeur) importable ailleurs
- C) Lancer le serveur

1. 6. CORS, en une idee :

- A) Un langage de programmation
- B) Une regle navigateur sur les requetes entre origines differentes
- C) Un type de variable

1. 7. Le debounce sert surtout a :

- A) Attendre un petit silence avant d'agir (ex: recherche)
- B) Chiffrer les mots de passe
- C) Remplacer CSS

1. 8. Ou placer le try/catch autour d'un chargement async ?

- A) Autour de l'appel await / logique de chargement
- B) Uniquement autour de console.log
- C) Autour de la balise html

1. 9. Pour charger un script module dans HTML, on ecrit souvent :

- A) `script type="module" src="main.js"`
- B) `script type="css" src="main.js"`
- C) `module href="main.js"`

1. 10. Un bon message d'erreur utilisateur ressemble a :

- A) `TypeError: undefined is not a function`
- B) Impossible de charger. Reessaie dans un instant.
- C) Une page blanche

1. 11. `response.json()` renvoie :

- A) Directement un nombre magique
- B) Une promesse qui donne les donnees parsees
- C) Un fichier PDF

1. 12. Organiser un projet, regle simple :

- A) Tout dans un seul fichier de 2000 lignes
- B) Un role clair par fichier
- C) Copier-coller le meme fetch partout

Corriges

1-B, 2-A, 3-B, 4-A, 5-B, 6-B, 7-A, 8-A, 9-A, 10-B, 11-B, 12-B.

Si tu as 9/12 ou plus : tu es pret pour les petits projets reels (formulaire, liste API, modules). Sinon, relis les chapitres lies (formulaire, erreurs reseau, modules, CORS), puis refais le quiz apres une semaine de pratique.

Petite histoire

Lea a rate `reponse.ok`. Elle a casse l'URL volontairement le soir meme et a vu le catch. Max a confondu `stringify` et `parse` : il a rejoue l'atelier JSON dix minutes. Sam affiche le score moyen sans nommer personne : "on revoit modules et CORS demain". Trois reactions, meme message : le quiz est une carte.

Questions bonus (auto-eval)

13. Cite deux ingredients obligatoires d'un fetch POST JSON.
14. Difference entre catch reseau et `!reponse.ok`.
15. Pourquoi tester en `file://` pose souvent probleme avec fetch et modules. Reponses libres : compare a tes chapitres 7, 8, 9.

Note de rythme

Prends le temps. DanielCraft forme des gens qui livrent. Si tu as rate des questions, c'est un plan de relecture, pas un echec. Refais le mini-projet, puis le quiz.

En vrai

Coche tes mauvaises reponses. Ouvre le chapitre lie. Refais un mini geste : un fetch casse, un `preventDefault`, un export. Cinq minutes. Un trou a la fois.

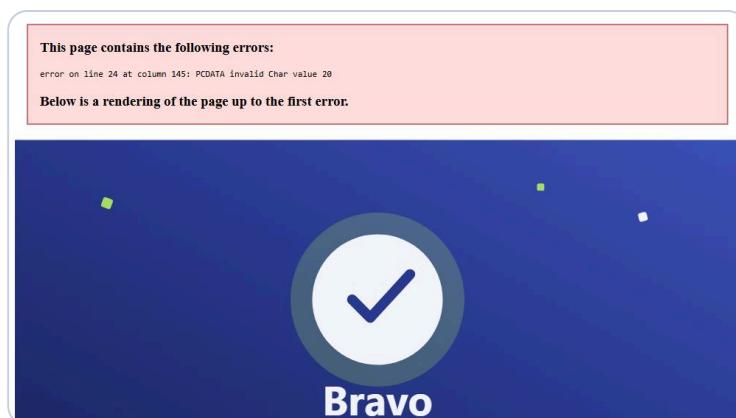
A toi

Ecris tes trois questions ratees. A cote, une phrase "ce que je confonds". Dans sept jours, rejoue seulement ces trois. Le vrai score, c'est la deuxieme passe.

Pour aller plus loin sans te perdre

Reviens a ce quiz dans un mois apres un vrai petit projet perso. Note ton score. L'objectif n'est pas 12/12 par coeur, c'est de ne plus bloquer sur fetch et JSON au quotidien.

Chapitre 21 - Bravo



Bravo. Tu sais faire parler ta page avec le monde extérieur.

Tu as parcouru un chemin solide : formulaires qui tiennent la route, JSON qui voyage, fetch GET et POST, promesses et async/await, erreurs réseau sans mensonge à l'utilisateur, modules pour découper, organisation de projet, débogage avec méthode, mini-projet liste API, trois ateliers, CORS sans mystère total, debounce et réflexes perf légers, et un quiz pour vérifier.

Ce n'est pas un diplôme magique. C'est mieux : des gestes concrets et une tête plus claire. Tu ne fais plus seulement réagir un bouton. Tu sais valider un formulaire, parler JSON, appeler un serveur, attendre proprement, découper en fichiers, et lire une erreur sans fondre. Chez DanielCraft, on considère que c'est ça, la vraie compétence JavaScript navigateur "suite" en 2026. Pas connaître vingt APIs par cœur. Savoir assembler le flux complet.

Lea confierait un petit ticket front junior après ce livre. Max peut maintenir son site sans copier-coller opaque. Sam corrige les exercices de ses élèves avec le même schéma mental. Toi, tu as la boucle : événement, validation, chargement, fetch, ok, json, affichage, catch. Garde-la.

Ce que tu sais faire maintenant

Charger des données depuis une URL. Afficher une liste dans le DOM. Dire à l'utilisateur quand ça charge et quand ça rate. Envoyer un POST JSON simple depuis un formulaire valide. Ranger ton code en api, afficher, main. Ralentir une recherche avec debounce. Comprendre pourquoi CORS bloque parfois. Débuguer avec console, Network et breakpoints. Tester volontairement l'échec. C'est déjà une posture pro.

La suite possible

Frameworks (React, Vue, Svelte...) plus tard, si tu veux. TypeScript pour typer. Tests automatiques. Bundlers (Vite, etc.). Backend Node ou PHP pour les vrais POST. Mais tu as le socle qui porte tout ça : fetch, async, structure, erreurs. Sans ce socle, un framework devient de la magie opaque. Avec, tu comprends ce qu'il automatise.

D'autres livres DanielCraft du parcours informatique approfondissent selon ta route : HTML/CSS suite, bases backend, ou IA pour accélérer le brouillon sans abdiquer la vérification. Pas ce soir si tu es fatigué. Digère. Puis choisis une suite.

Un dernier rappel

Le reseau ne repond pas toujours. Ce n'est pas personal. Verifie ok. Message humain. console.error pour toi. Decoupe tot. Un mini-projet livre vaut dix tutos consommes. Garde le gouvernail : tu pilotes, le code propose, l'utilisateur merite la verite quand ca casse.

Engagement 30 jours

Chaque semaine : une mini-feature (liste, contact, recherche debounced). Chaque fin de semaine : relis le chapitre retenir. J+30 : refais le quiz et le mini-projet sur une nouvelle source de donnees. Tu transformeras ce livre en muscle. C'est le vrai bravo.

♦ ASTUCE

Bloque quatre creneaux de 45 minutes maintenant - un par semaine. Sans creneau, l'engagement reste une intention de dimanche soir.

Petite scene DanielCraft

Lea ouvre un ticket "afficher les avis clients". Elle valide le formulaire, fetch les avis, catch propre, modules propres. Quarante minutes, livrable clair. Max ajoute la meteo sur sa page d'accueil sans copier-coller un tuto opaque : il reconnaît le schema. Sam prepare le prochain exercice avec citations.json et atelier modules. Trois profils, un meme socle. Aucun n'a le code parfait. Tous ont livre.

En vrai

Ouvre ton editeur maintenant. Cree un dossier projet-perso. Ecris index.html vide et un main.js avec juste console.log("suite"). Commit mental : tu as demarre. Dix minutes battent une intention de "je commencerai demain".

A toi

Ecris en trois lignes : ton prochain projet perso, la donnee que tu vas charger ou envoyer, et le premier fichier que tu vas creer (souvent index.html ou main.js). Puis ouvre l'editeur. Pas demain. Maintenant, meme dix minutes.

Bravo. Vraiment. Va livrer quelque chose de petit, verifie les deux chemins succes et erreur, et range ton code.

Note de rythme

Prends le temps de celebrer, puis enchaîne avec une action minuscule butee. DanielCraft forme des gens qui livrent, meme petit. C'est ta graduation : un fichier de plus dans le monde, pas une page de plus dans un livre.

Zoom : le schema mental

Evenement, validation, chargement, fetch, ok, json, affichage, catch. Si tu peux dire cette chaine a voix haute, tu as le livre dans la tete. Lea la repete aux juniors. Max la murmure avant chaque ticket. Sam la met au tableau. Toi, tu la gardes sur un post-it. Le vrai bravo, c'est quand tu la retrouves sans ouvrir le PDF.

Tu as les briques. Maintenant, construis.